

파이썬 기초: 기초 문법

#파이썬이란? #환경설정 #기초자료형 #연산자

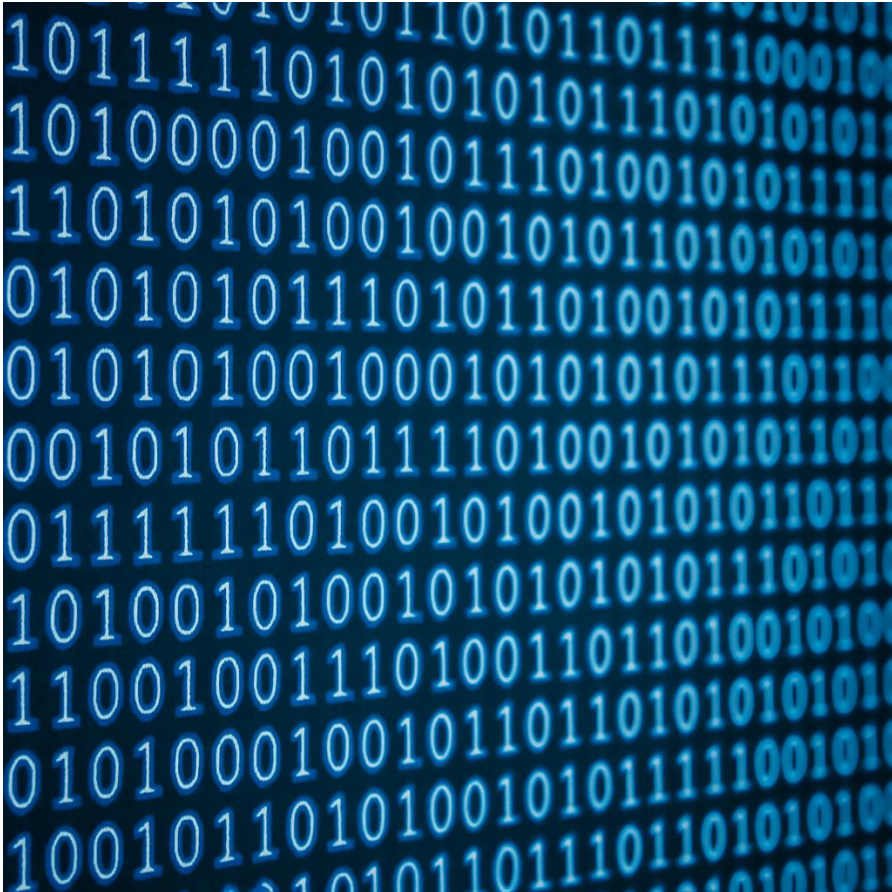
목차

- 01 파이썬이란?
- 02 Python 환경설정
- 03 기초 자료형과 출력
- 04 변수
- 05 자료형의 연산
- 06 입력과 형 변환
- 07 논리 자료형과 비교/논리 연산자



파이썬이란?

프로그래밍 언어(Programming Language)란?



- 사용자가 컴퓨터에게 명령을 해야 동작을 함.
- 하지만, 컴퓨터는 0과 1로 이루어진 기계어로 구성됨.
→ 사람의 언어를 이해하지 못함.

컴퓨터와 사람 간의 서로 소통하기 위한 언어



파이썬(Python)

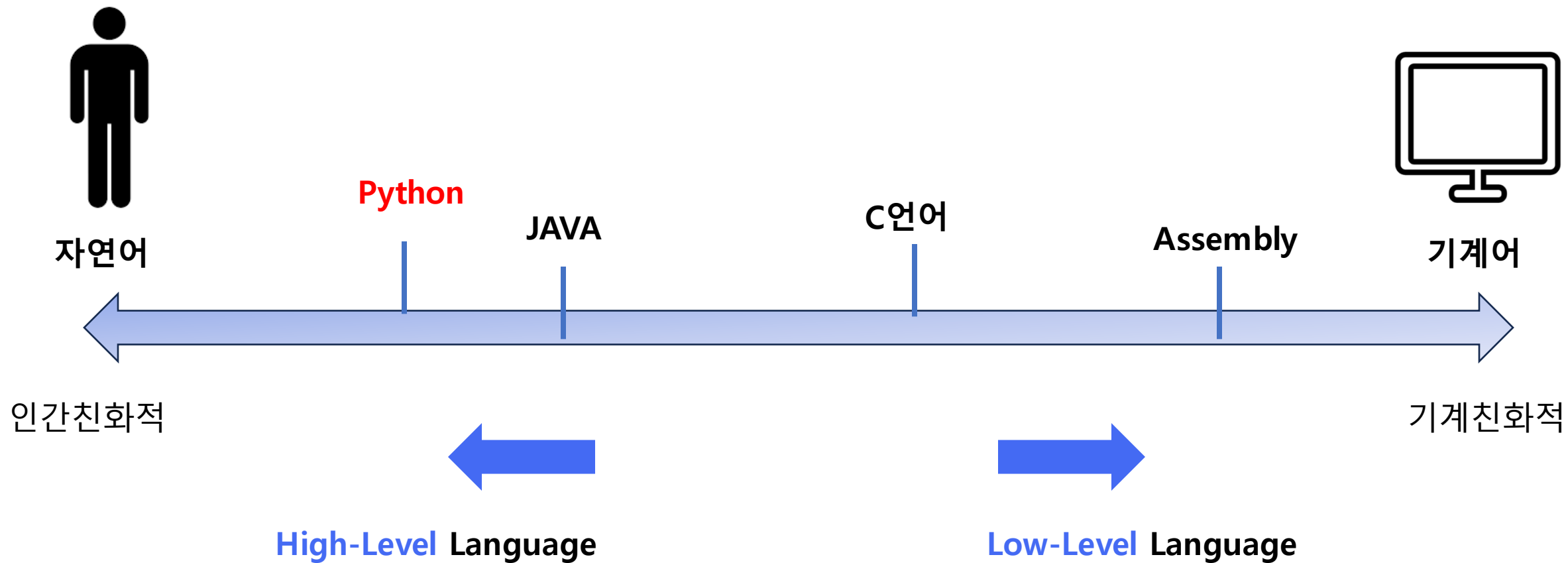


Guido Van Rossum

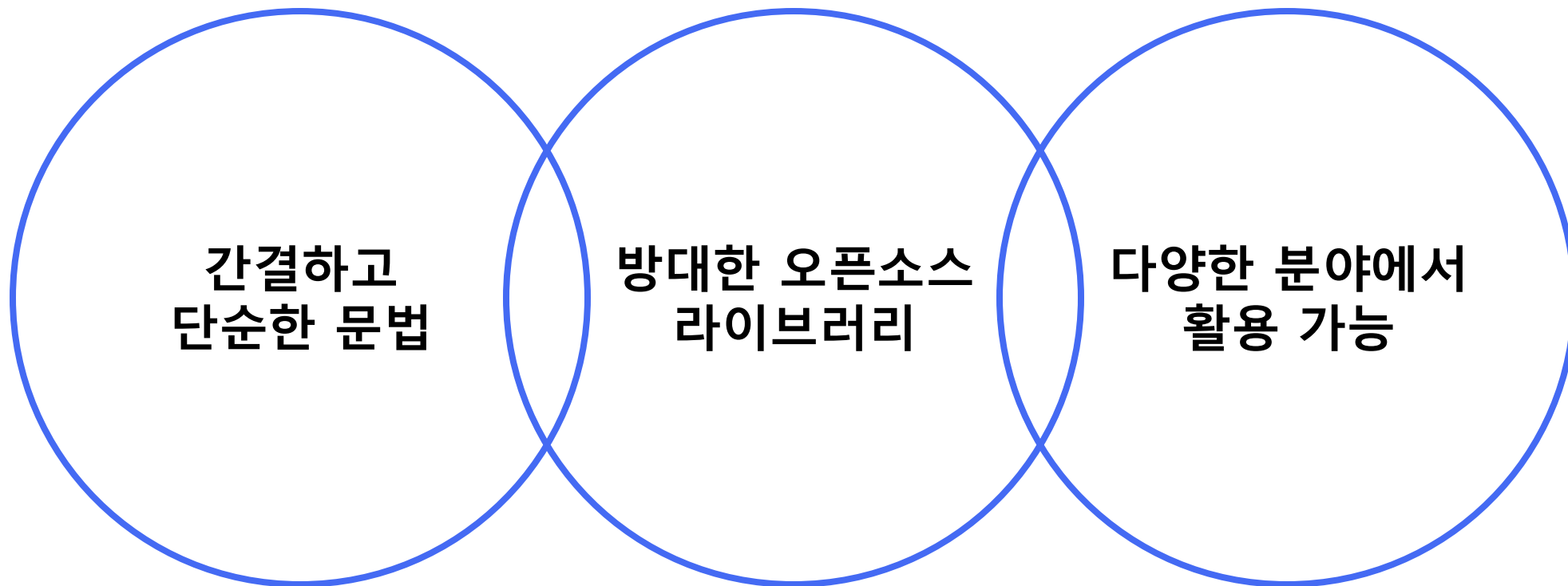
1991년 첫번째 릴리즈

파이썬이란?

언어의 수준



Why 파이썬?



파이썬이란?

Why 파이썬? (1) 간결하고 단순한 문법



```
print("Hello World")
```



```
#include <stdio.h>

int main(){
    printf("Hello World");
    return 0;
}
```



```
public class Main{

    public static void main(String[] args){
        System.out.println("Hello World");
    }

}
```


Why 파이썬? (2) 방대한 오픈 소스 라이브러리

679,341 projects

7,415,642 releases

15,504,917 files

958,050 users

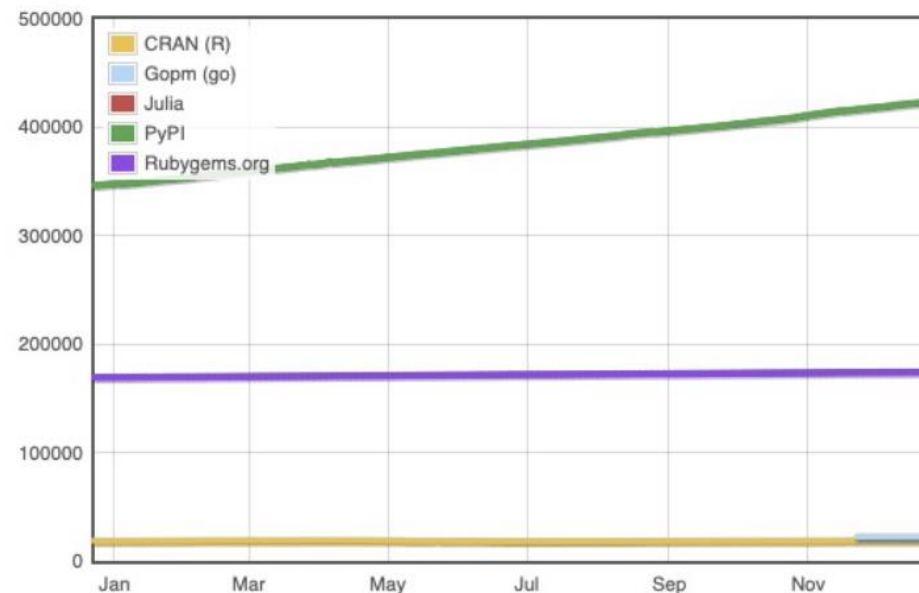


The Python Package Index (PyPI) is a repository of software for the Python programming language.

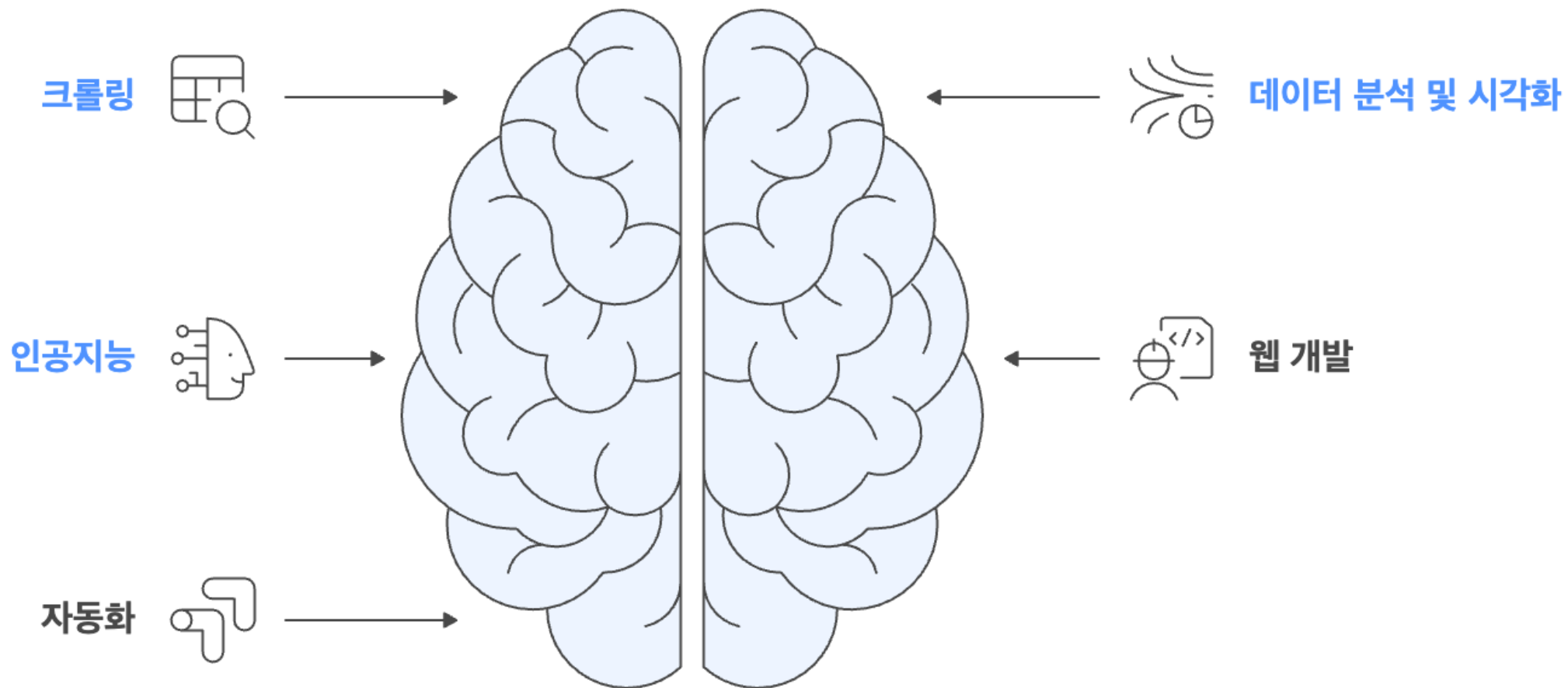
PyPI helps you find and install software developed and shared by the Python community. [Learn about installing packages](#).

Package authors use PyPI to distribute their software. [Learn how to package your Python code for PyPI](#).

Module Counts



Why 파이썬? (3) 다양한 분야에서 활용 가능





Python 환경설정

설치 항목



Mini-Conda

Python 실행환경

+

Conda 가상환경

+

pip 패키지 관리자



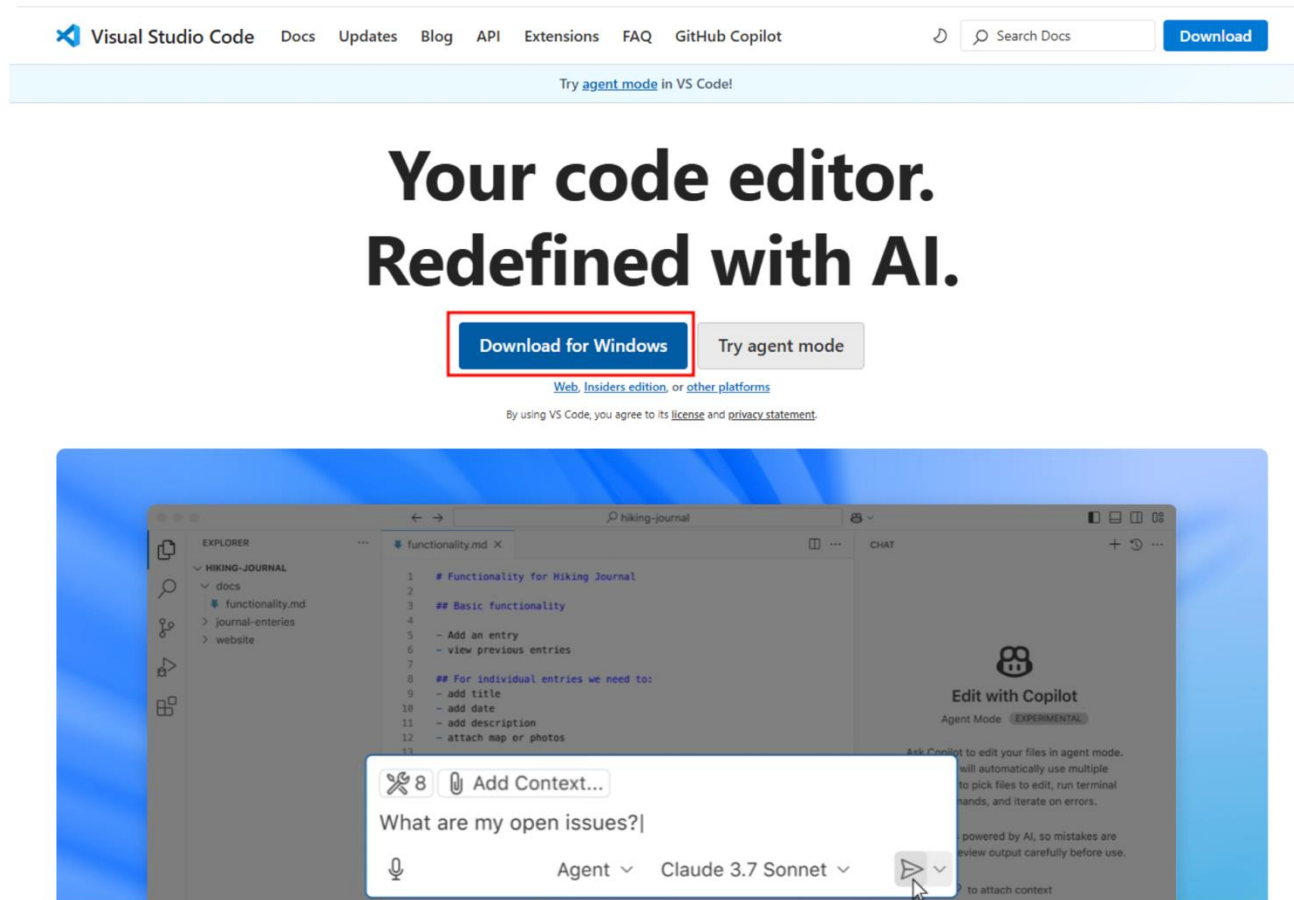
Visual Studio Code

Visual Studio Code

코드 에디터(IDE)

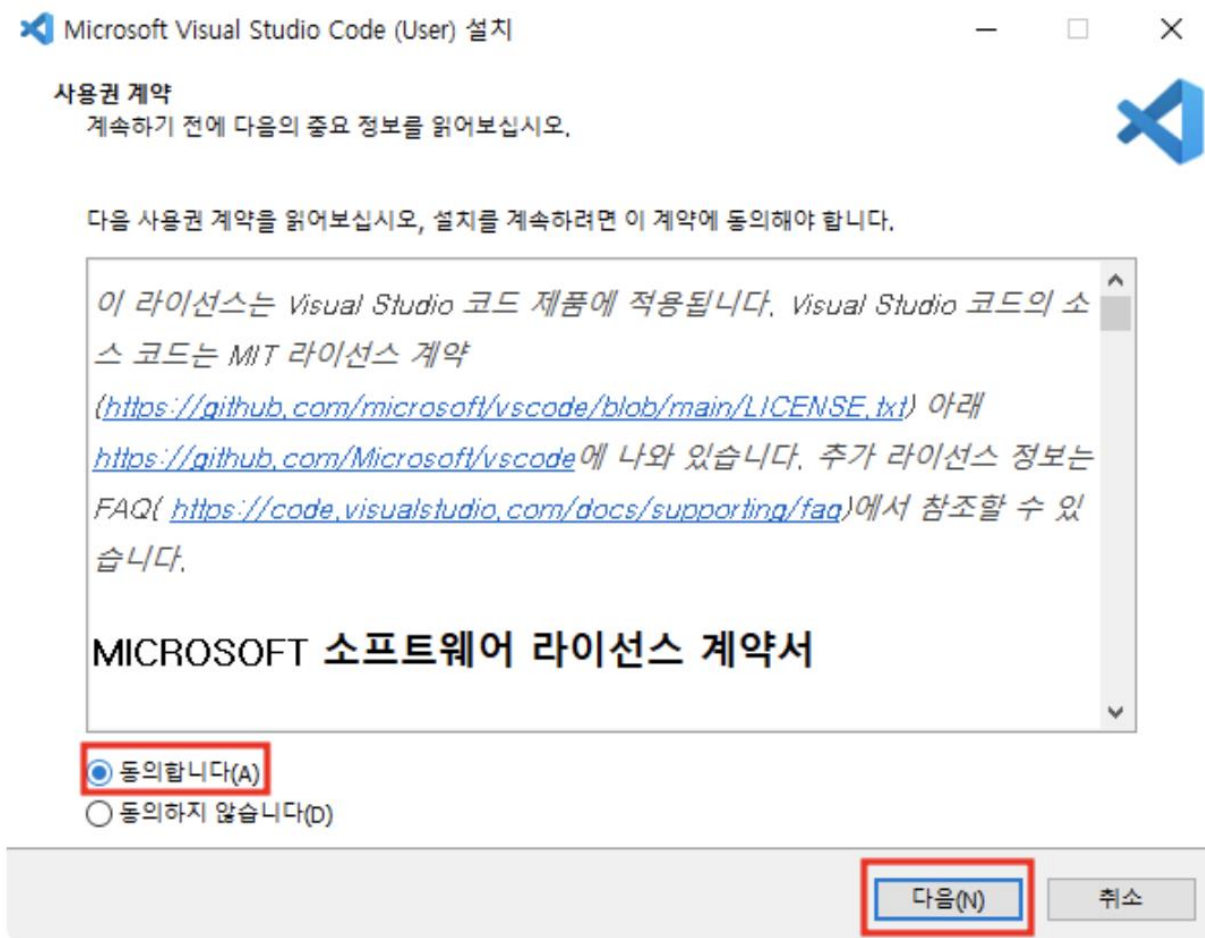
코드 작성/실행/디버깅/확장 등

(1) Visual Studio Code 설치하기

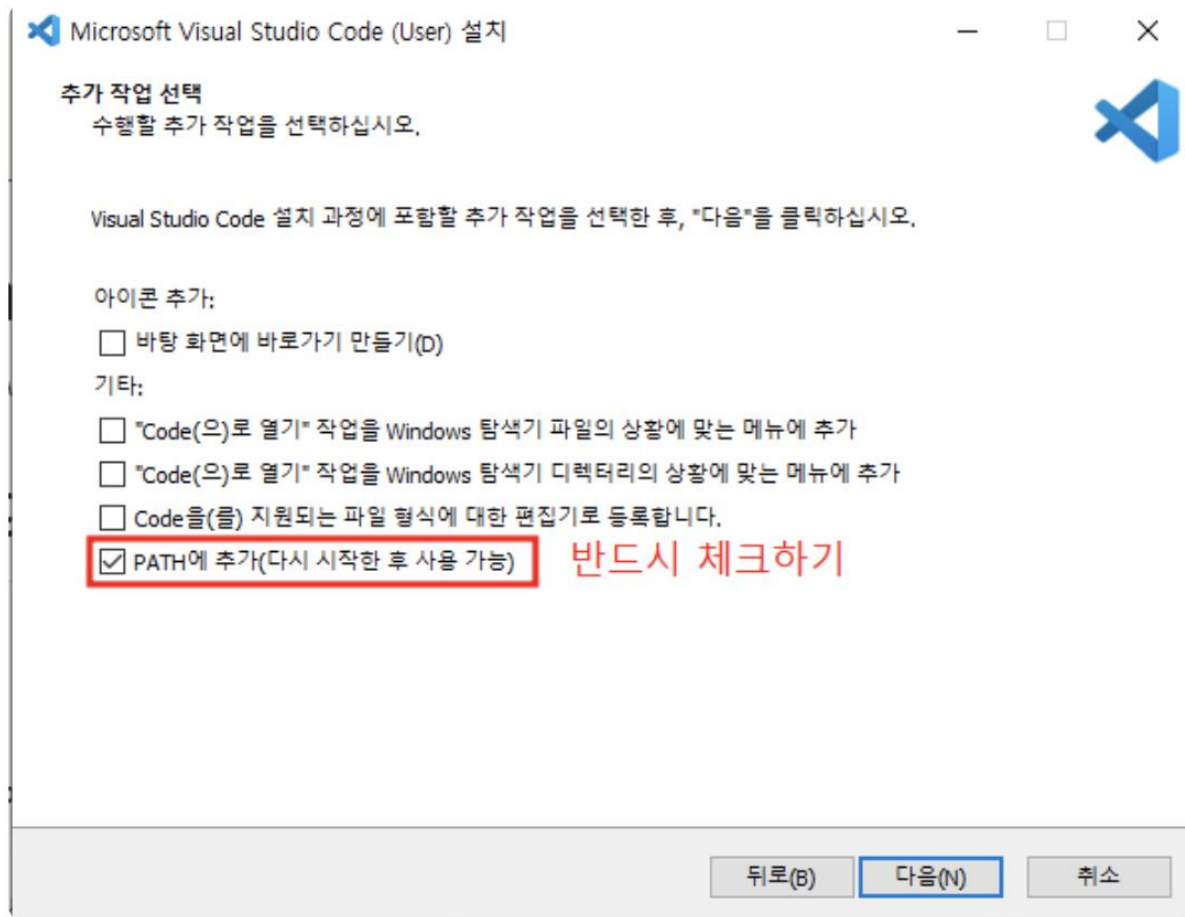


<https://code.visualstudio.com/>

(1) Visual Studio Code 설치하기

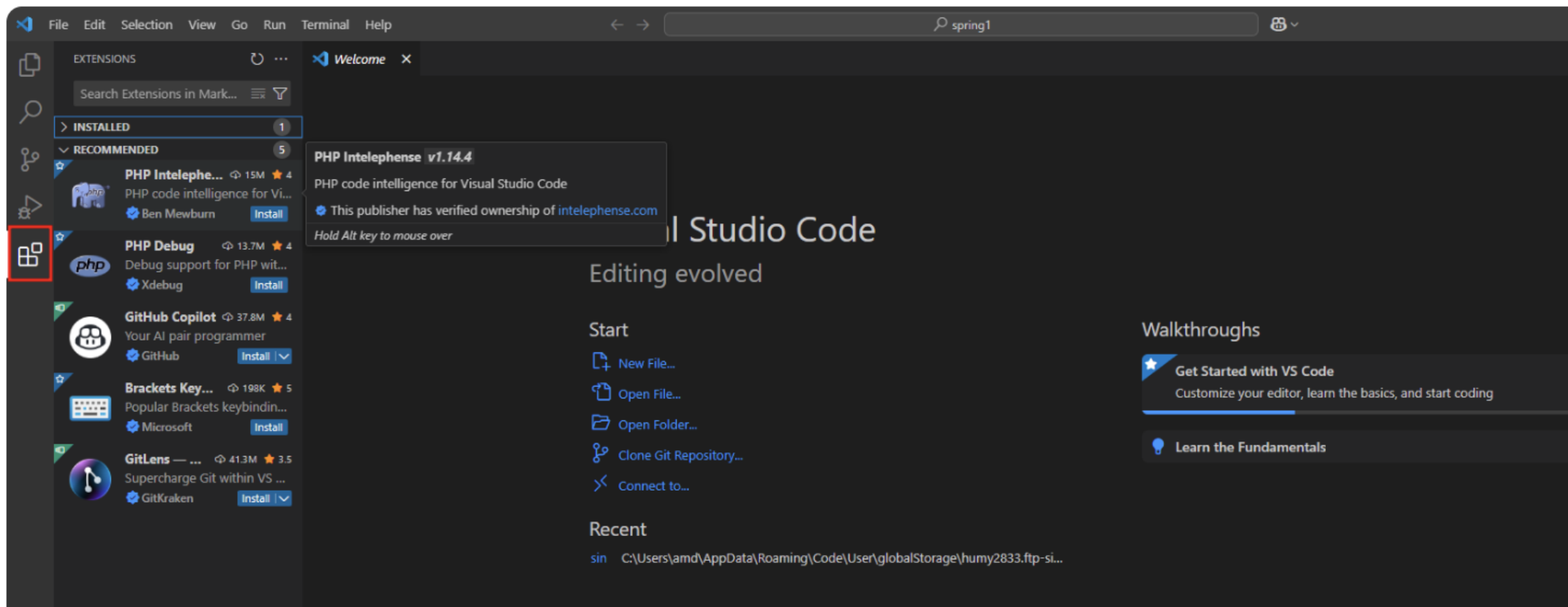


(1) Visual Studio Code 설치하기



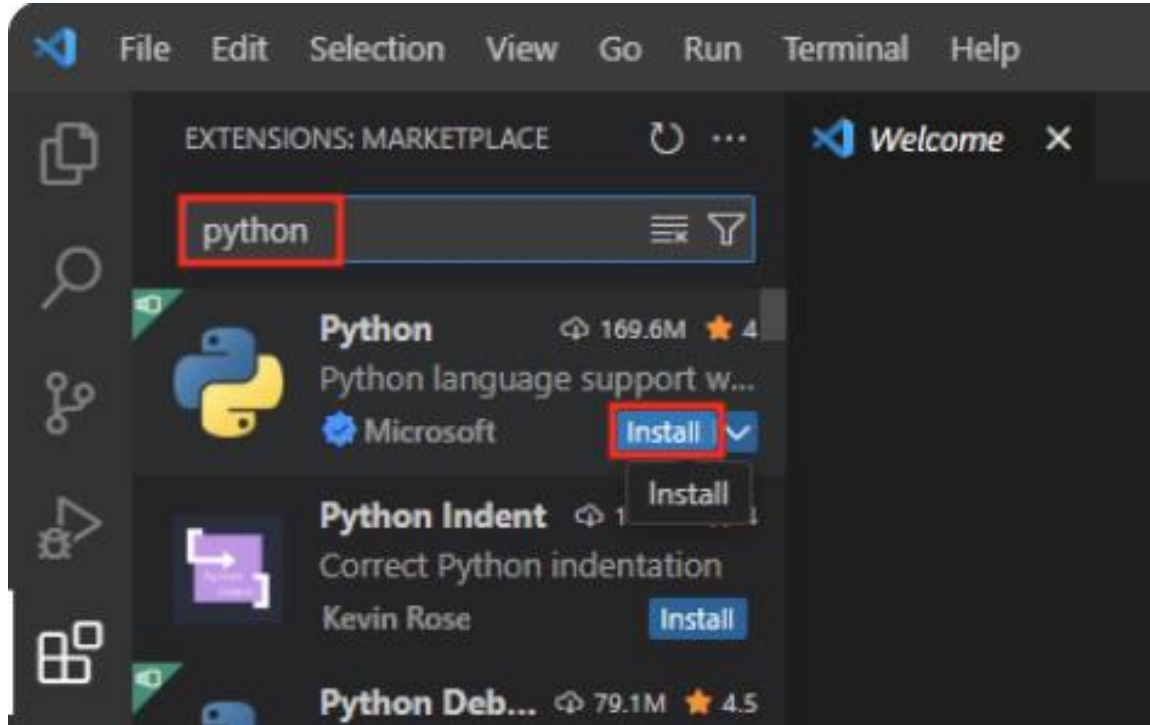
다음 항목에서 '**PATH에 추가(다시 시작한 후 사용 가능)**' 항목을 반드시 체크해주시고 '다음' 버튼을 클릭하고, 이어서 다음으로 특별한 설정 없이 계속 넘어간 뒤에 설치를 완료해줍니다.

(1) Visual Studio Code 열고 Extension 설치



VS Code에서 Python 코드를 작성할 때 다양한 도구와 기능을 제공해줄 수 있는 파이썬 전용 확장을 설치
VS Code 첫 화면이 위와 비슷할텐데, 여기서 왼쪽 탭 메뉴에서 맨 아래에 있는 'Extensions' 탭으로 이동

(1) Visual Studio Code 열고 Extension 설치



여기서 검색창에 'Python'을 검색한 뒤,
제일 첫번째로 나오는 항목인 'Python' 항목에 대해 'Install' 버튼을 클릭하여 설치

(2) MiniConda 설치하기

The screenshot shows the Anaconda website's download page. The navigation bar includes the Anaconda logo, links for Products, Solutions, Resources, and Company, and buttons for Free Download, Sign In, and Get Demo. A banner for 'Students & educators' is visible. The main content area features a large 'Download Now' button and text instructing users to download Anaconda Distribution or Miniconda. Below this, there are two sections: 'Distribution Installers' and 'Miniconda Installers'. The 'Miniconda Installers' section is highlighted with a red rectangle, and its 'Download' button is also highlighted. A download bar on the right side of the browser shows the file 'Miniconda3-latest-Windows-(1).exe' (87.9MB) is complete. A chatbot icon is visible in the bottom right corner.

ANACONDA Products Solutions Resources Company Free Download Sign In Get Demo

Students & educators Enjoy free access to Anaconda's premium AI tools. Learn More

Home / Free Download

Download Now

Download Anaconda Distribution or Miniconda by choosing the proper installer for your machine. Learn the difference from our [Documentation](#).

Distribution Installers

Download

For installation assistance, refer to [troubleshooting](#).

Miniconda Installers

Download

For installation assistance, refer to [troubleshooting](#).

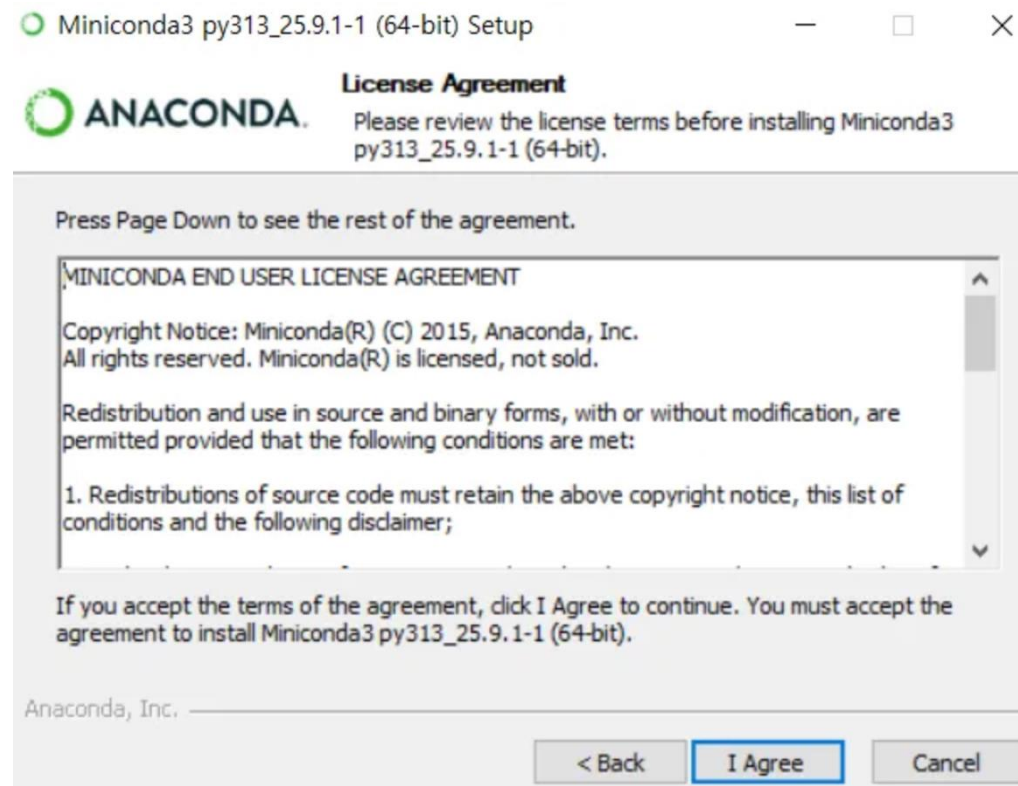
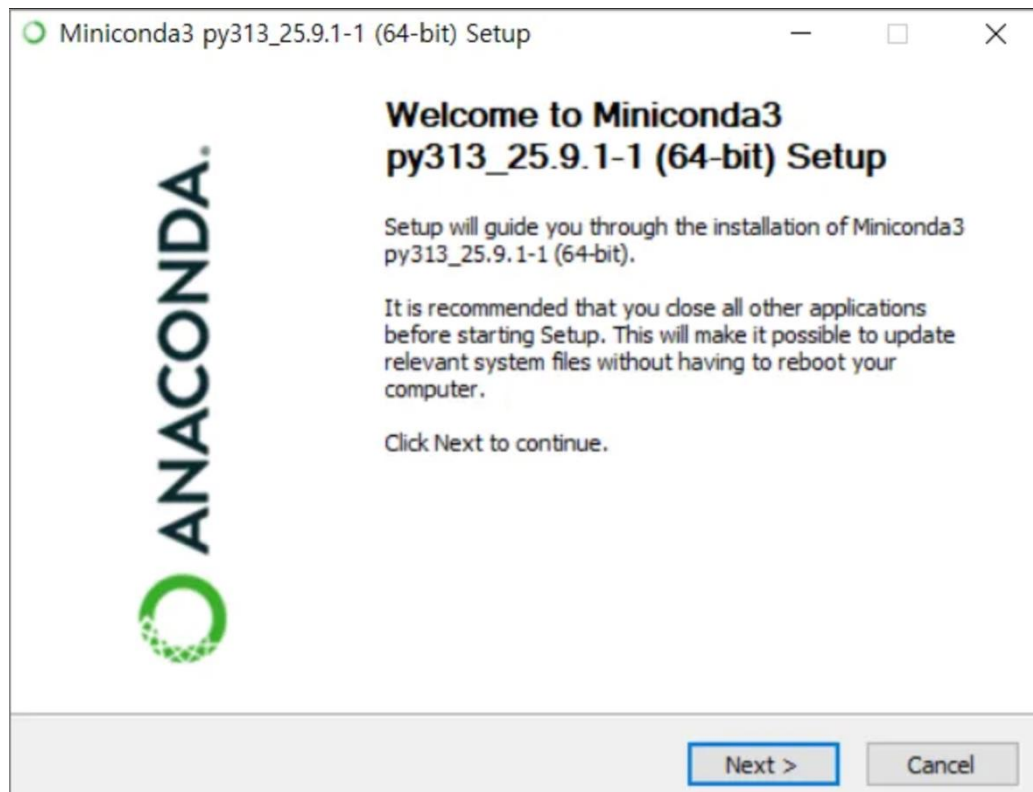
Hi, how can I help?

최근 다운로드 기록

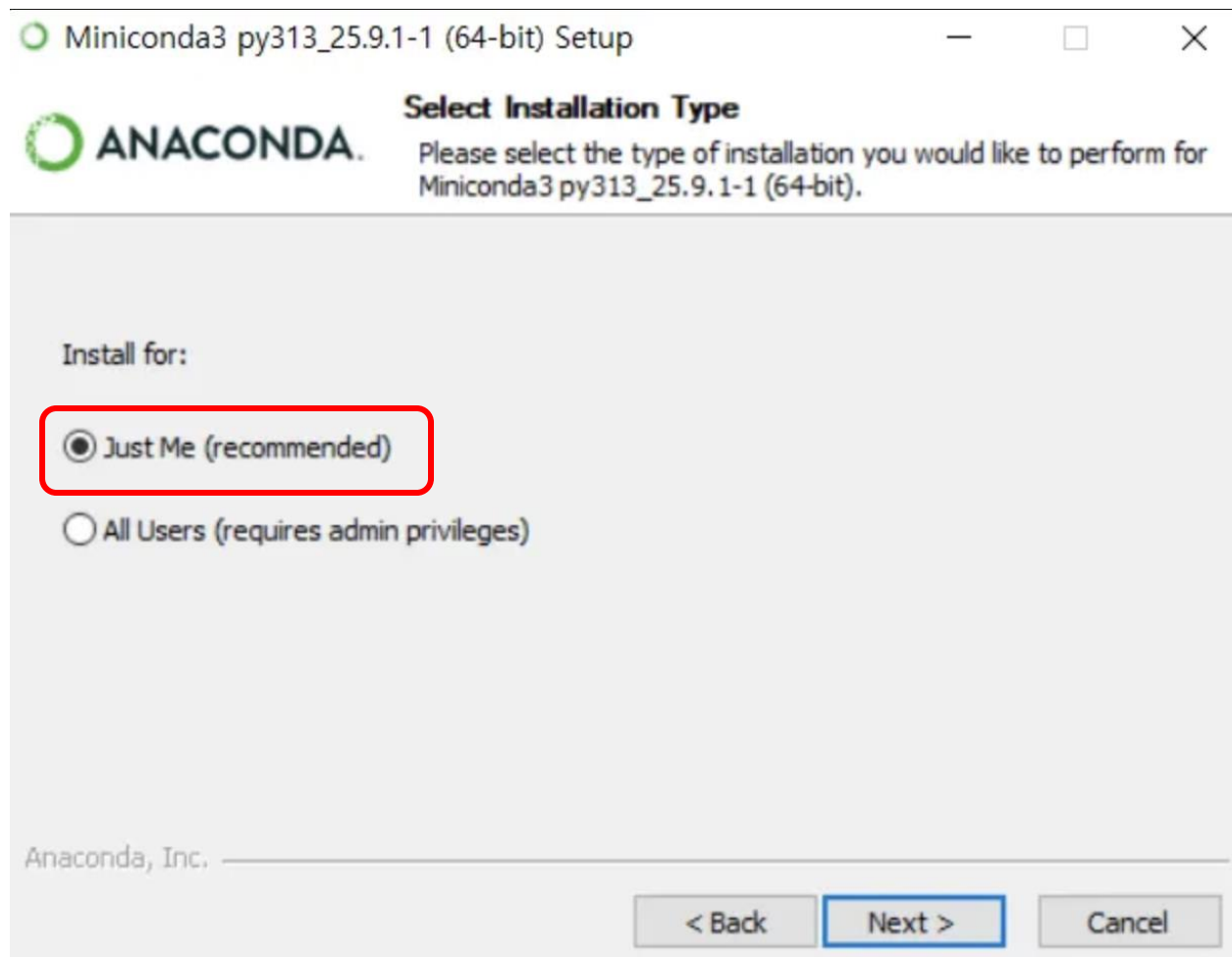
Miniconda3-latest-Windows-(1).exe
87.9MB • 완료

<https://www.anaconda.com/download/success>

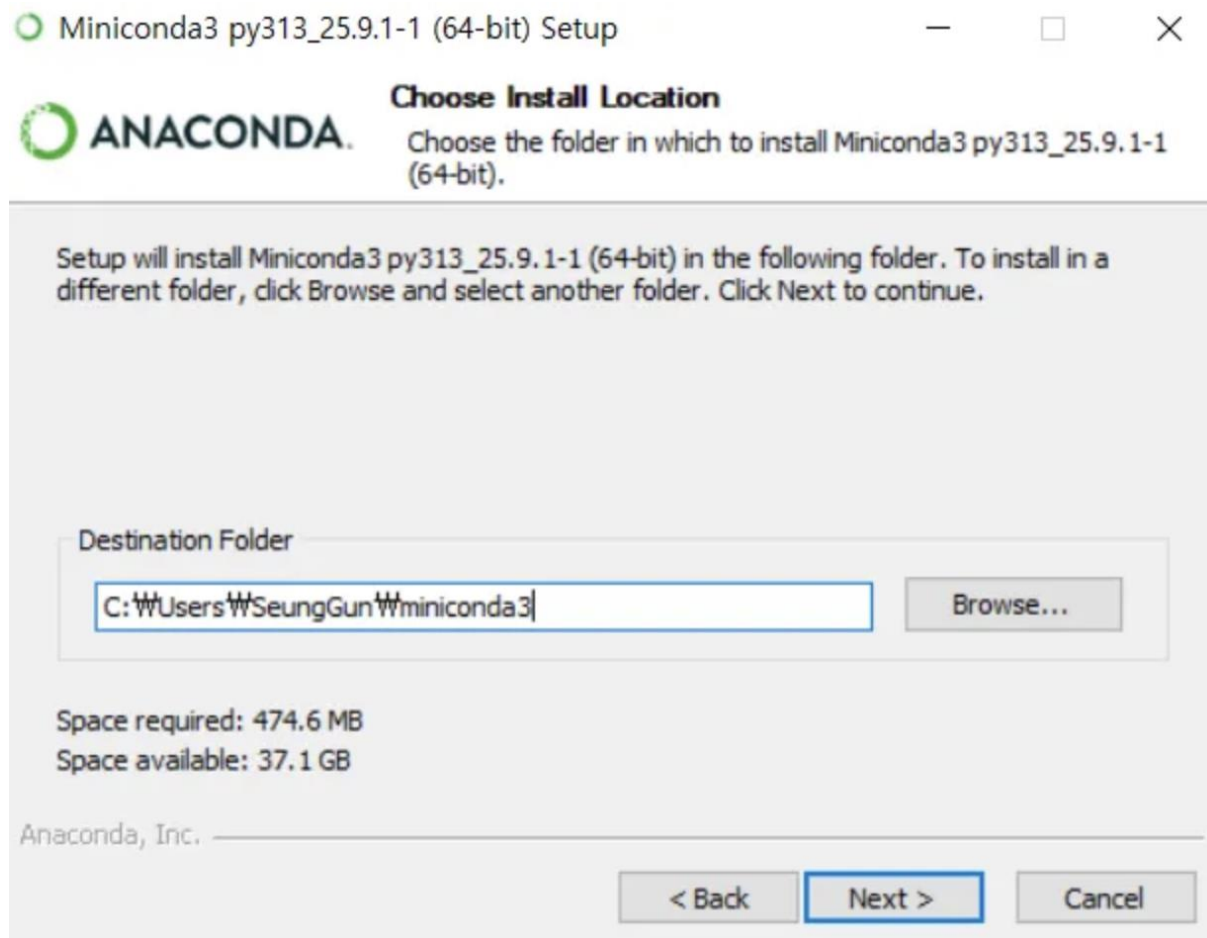
(2) MiniConda 설치하기



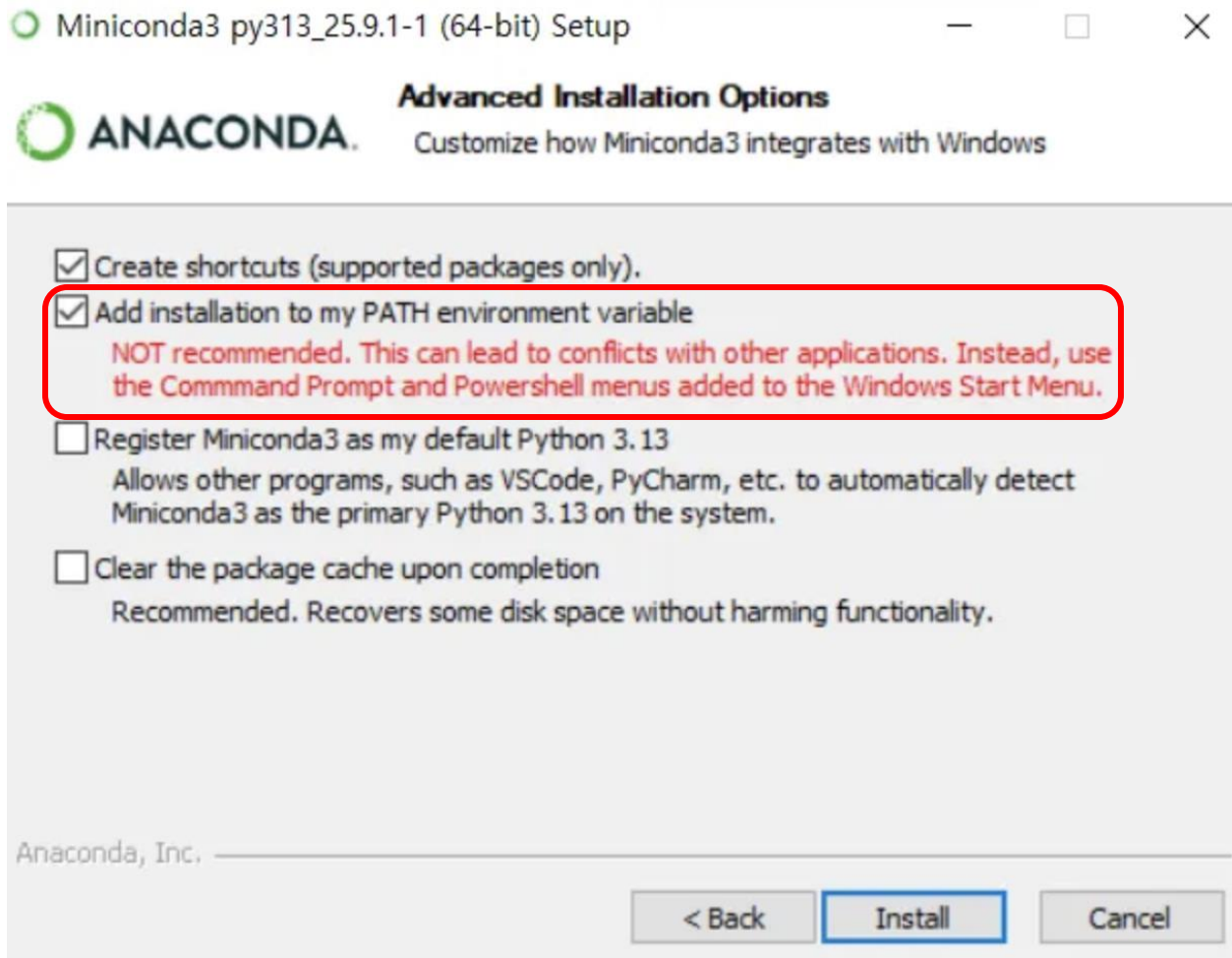
(2) MiniConda 설치하기



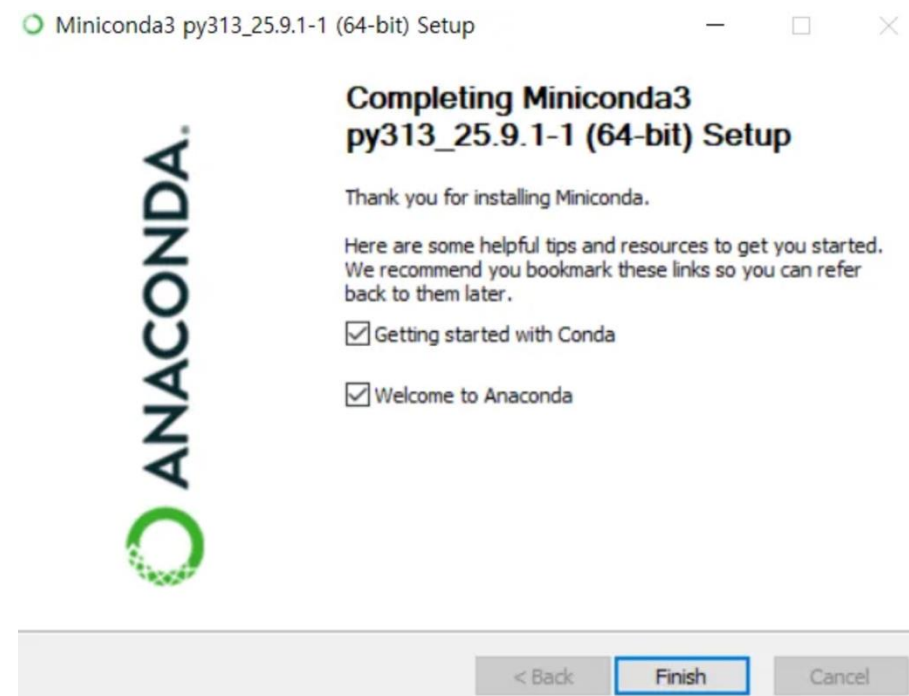
(2) MiniConda 설치하기



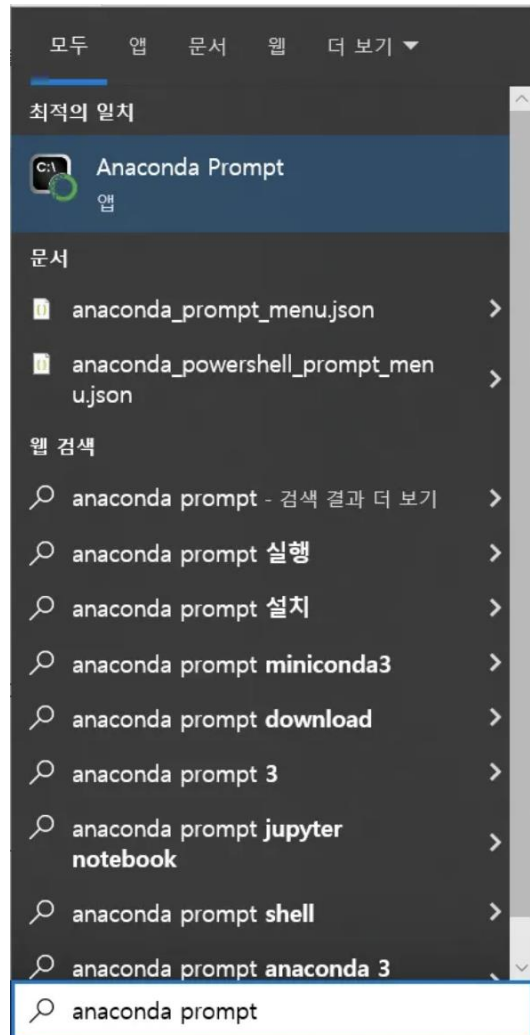
(2) MiniConda 설치하기



체크하기!



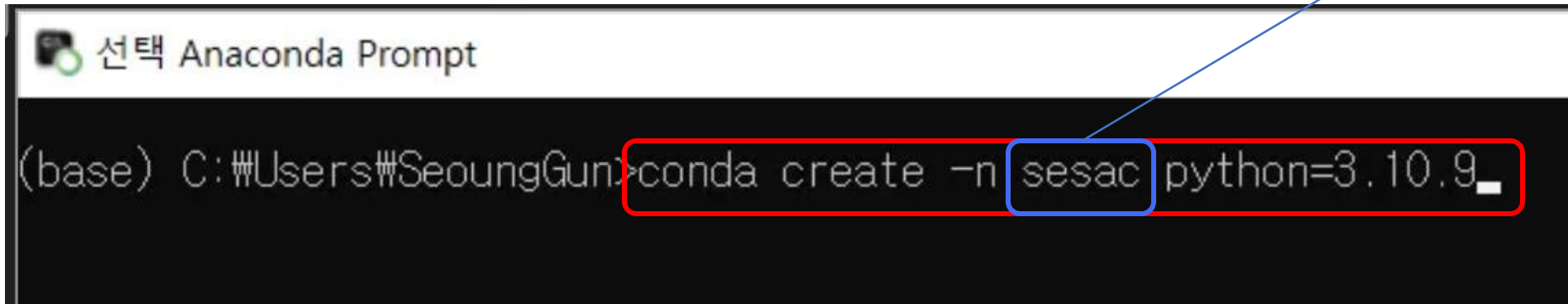
(3) MiniConda 가상환경 설정하기



윈도우 검색에서 “anaconda prompt” 클릭

(3) MiniConda 가상환경 설정하기

가상환경 이름(자유롭게 설정)



```
선택 Anaconda Prompt
(base) C:\Users\SeoungGun>conda create -n sesac python=3.10.9_
```

The screenshot shows a terminal window titled '선택 Anaconda Prompt'. The command prompt is '(base) C:\Users\SeoungGun>'. The command being entered is 'conda create -n sesac python=3.10.9_'. The text 'sesac' is highlighted with a blue box, and a blue arrow points from the text '가상환경 이름(자유롭게 설정)' to this box. The entire command is enclosed in a red rectangular box.

파이썬 3.10.9 버전으로 가상환경을 생성하는 명령어

(3) MiniConda 가상환경 설정하기

```
## Package Plan ##

environment location: C:\Users\SeoungGun\miniconda3\envs\sesac

added / updated specs:
- python=3.10.9

The following packages will be downloaded:

package | build | size
-----|-----|-----
openssl-1.1.1w | h2bfff1b_0 | 5.5 MB
python-3.10.9 | h966fe2a_2 | 15.8 MB
setuptools-80.9.0 | py310haa95532_0 | 1.4 MB
wheel-0.45.1 | py310haa95532_0 | 145 KB
Total: 22.8 MB

The following NEW packages will be INSTALLED:

bzip2 pkgs/main/win-64::bzip2-1.0.8-h2bfff1b_6
ca-certificates pkgs/main/win-64::ca-certificates-2025.9.9-haa95532_0
libffi pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
libzlib pkgs/main/win-64::libzlib-1.3.1-h02ab6af_0
openssl pkgs/main/win-64::openssl-1.1.1w-h2bfff1b_0
pip pkgs/main/noarch::pip-25.2-pyh0872135_1
python pkgs/main/win-64::python-3.10.9-h966fe2a_2
setuptools pkgs/main/win-64::setuptools-80.9.0-py310haa95532_0
sqlite pkgs/main/win-64::sqlite-3.50.2-hda9a48d_1
tk pkgs/main/win-64::tk-8.6.15-hf199647_0
tzdata pkgs/main/noarch::tzdata-2025b-h04d1e81_0
ucrt pkgs/main/win-64::ucrt-10.0.22621.0-haa95532_0
vc pkgs/main/win-64::vc-14.3-h2df5915_10
vc14_runtime pkgs/main/win-64::vc14_runtime-14.44.35208-h4927774_10
vs2015_runtime pkgs/main/win-64::vs2015_runtime-14.44.35208-ha6b5a95_10
wheel pkgs/main/win-64::wheel-0.45.1-py310haa95532_0
xz pkgs/main/win-64::xz-5.6.4-h4754444_1
zlib pkgs/main/win-64::zlib-1.3.1-h02ab6af_0

Proceed ([y]/n)? y
```

y 입력

```
done
#
# To activate this environment, use
#
#     $ conda activate sesac
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\SeoungGun>
```

생성 완료 후

conda activate [가상환경 이름]:
해당 가상환경 활성화

conda deactivate:
현재 활성화 중인 가상환경 비활성화

(3) MiniConda 가상환경 설정하기

```
(base) C:\Users\SeoungGun>conda activate sesac  
(sesac) C:\Users\SeoungGun>_
```

활성화된 가상환경
(설정된 이름으로 뜨면 성공)

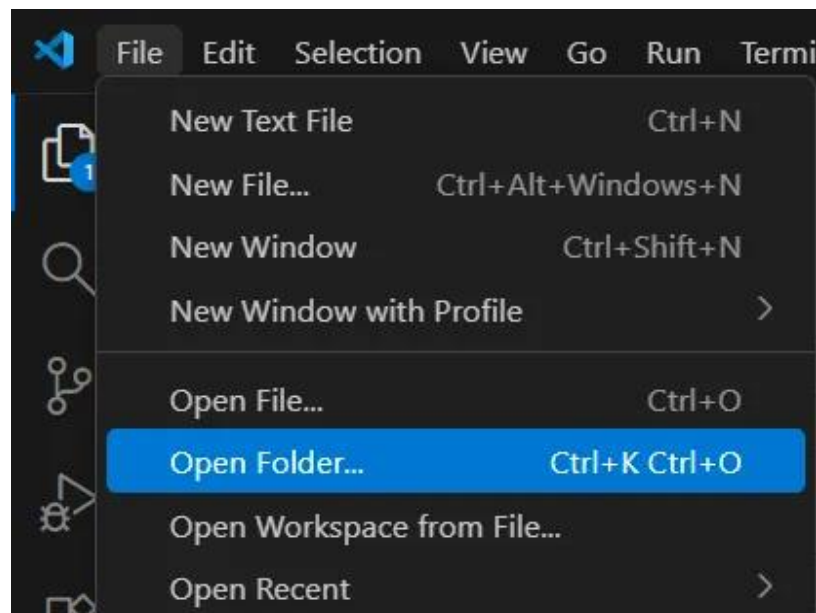
생성한 가상환경 활성화하기

(4) MiniConda + VSCode 연결하기

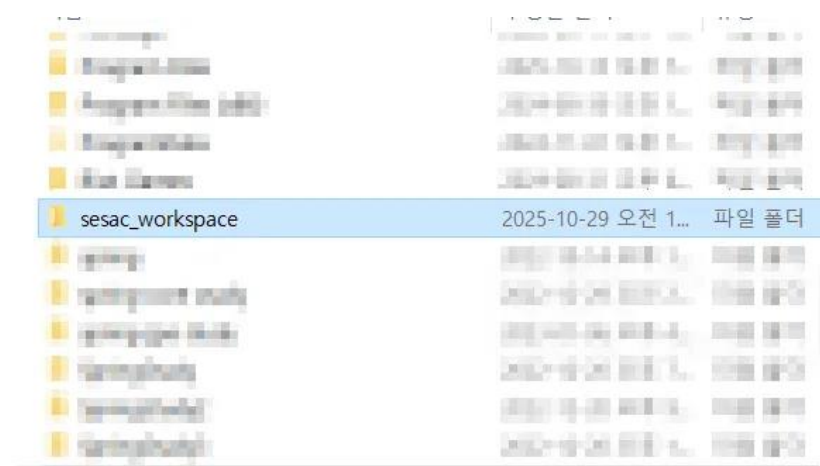
워크스페이스(폴더) 생성 후 VSCode에서 열기

sesac_workspace

이름은 자유롭게
(또는 새싹_본인이름)

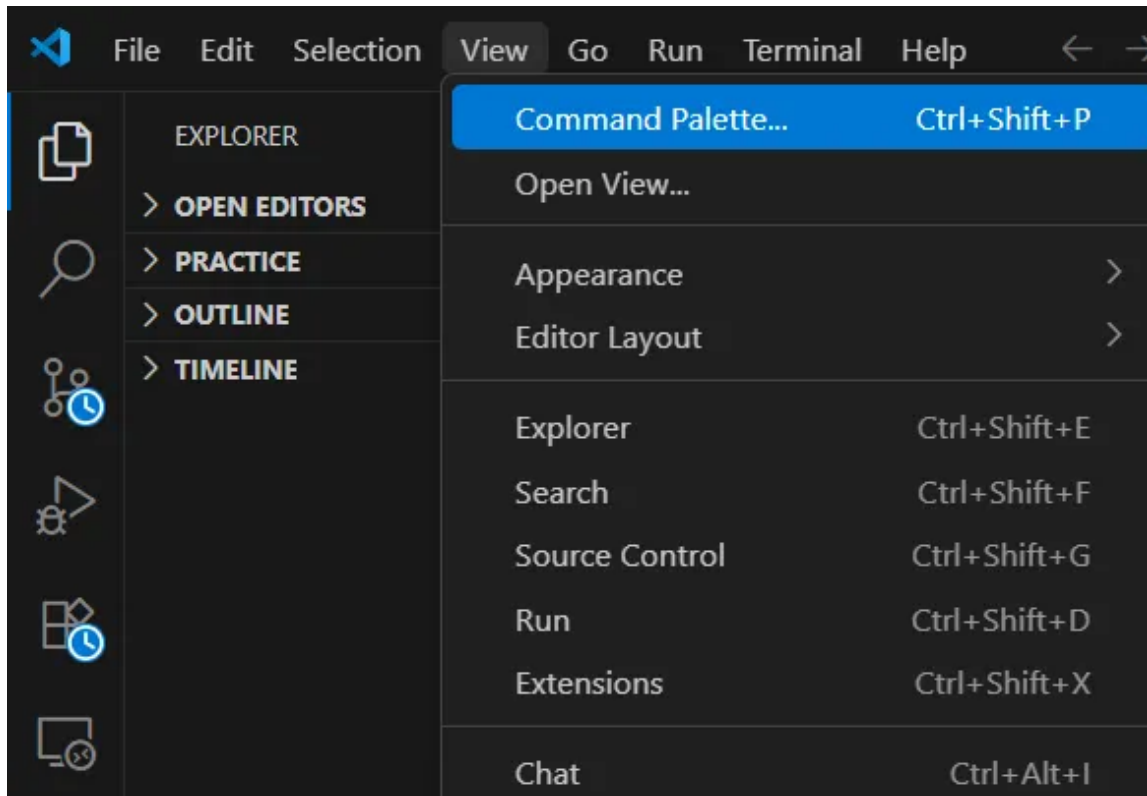


File > Open Folder



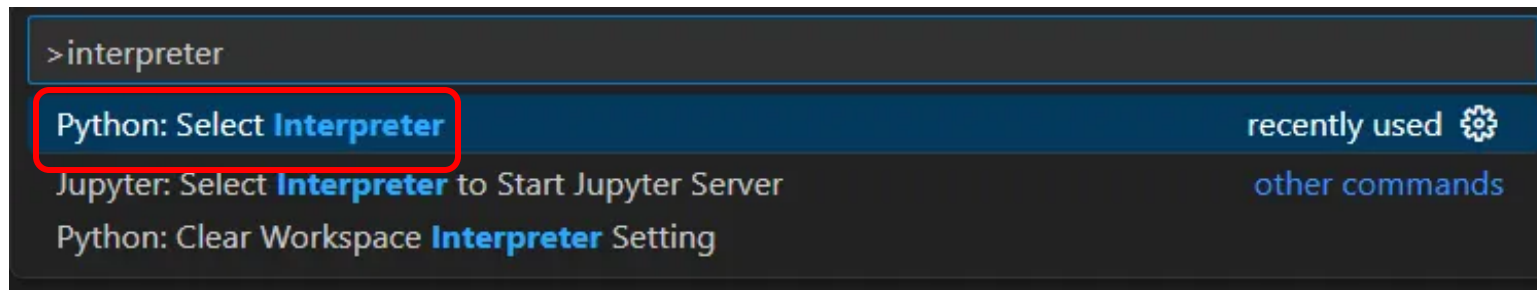
생성한 폴더 선택

(4) MiniConda + VSCode 연결하기

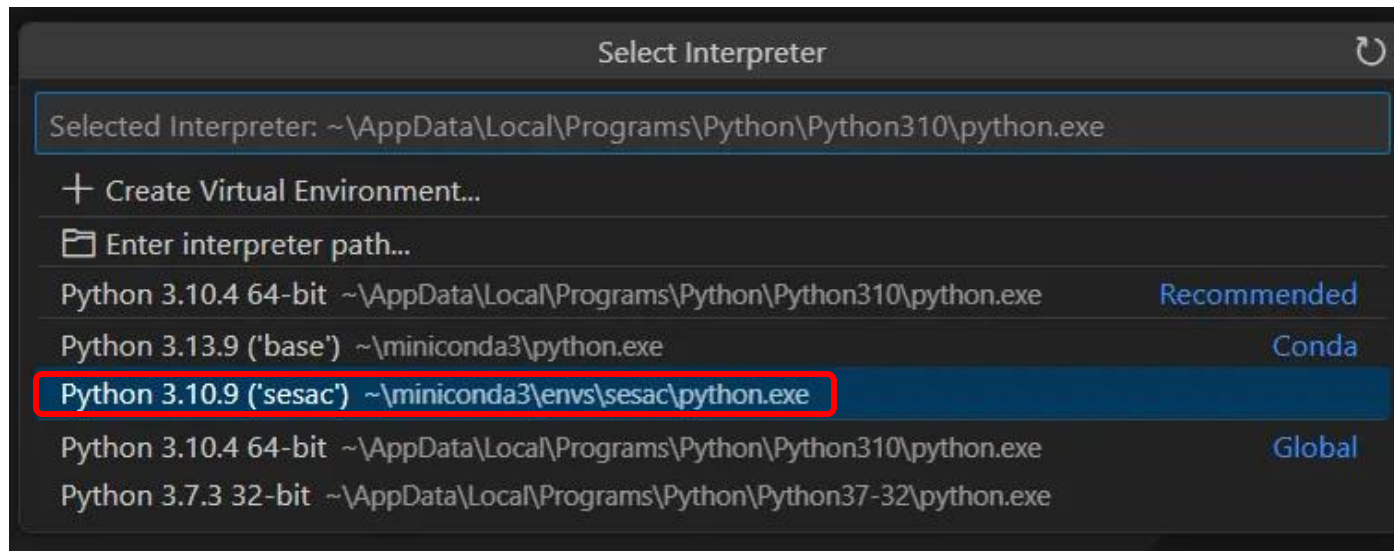


View > Command Palette 열기
(또는 Ctrl + Shift + P)

(4) MiniConda + VSCode 연결하기



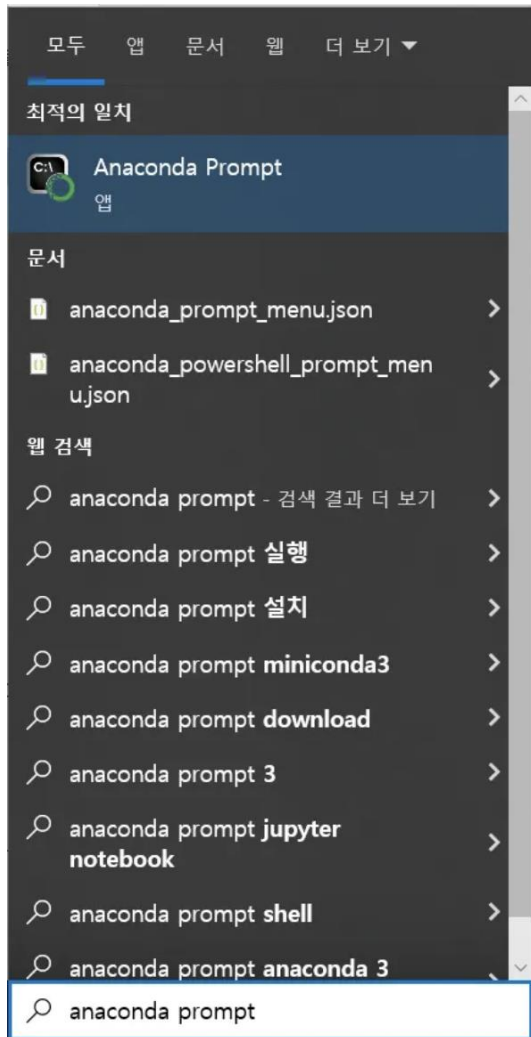
가운데 상단에 뜨는 입력 창에서
Python: Select Interpreter 검색 후
선택



생성한 가상환경 이름에 맞는 파이썬 선택

Python [버전](가상환경이름)

(4) MiniConda + VSCode: 주피터 노트북 환경 설정(실습 환경 구축)



Anaconda Prompt 다시 접속 후 아래 3개 명령어 입력

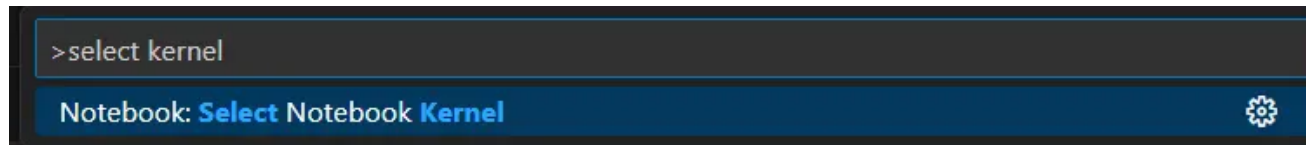
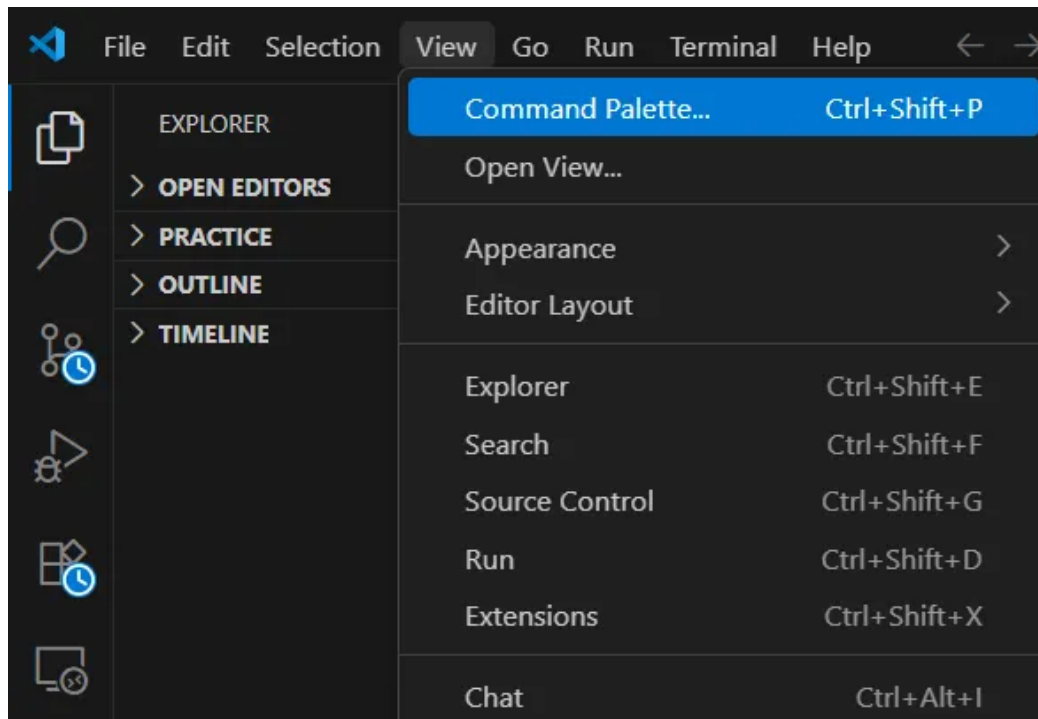
```
(sesac) C:\Users\SeoungGun>pip install jupyter
```

```
(sesac) C:\Users\SeoungGun>pip install ipykernel
```

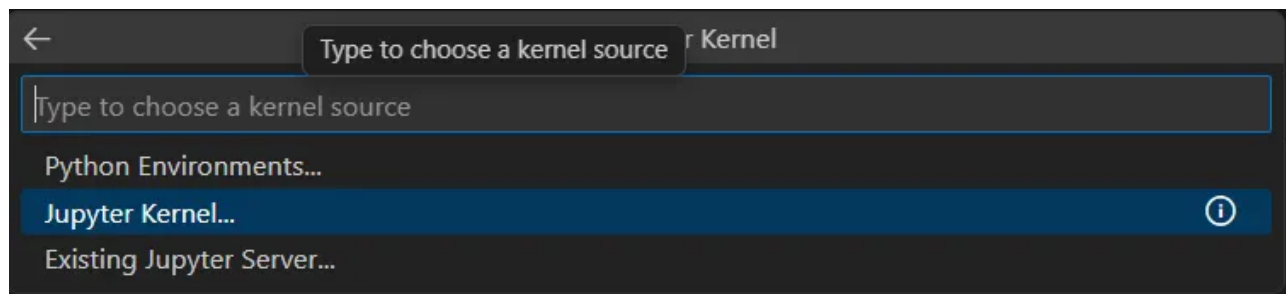
```
(sesac) C:\Users\SeoungGun>python -m ipykernel install --user --name sesac --display-name "sesac_kernel"
```

```
python -m ipykernel install --user --name [가상환경이름] --display-name "커널이름"
```

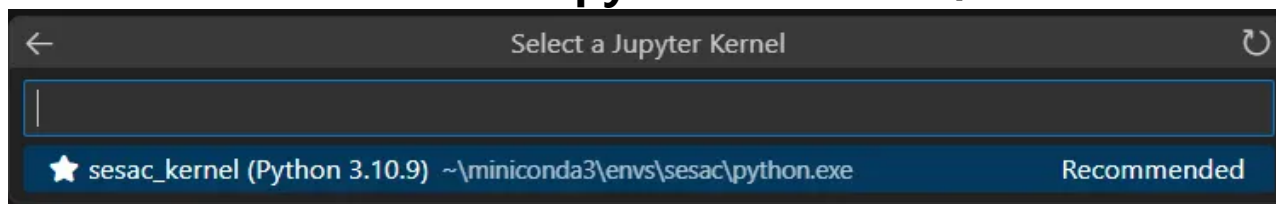
(4) MiniConda + VSCode: 주피터 노트북 환경 설정(실습 환경)



Notebook: Select Notebook Kernel 검색

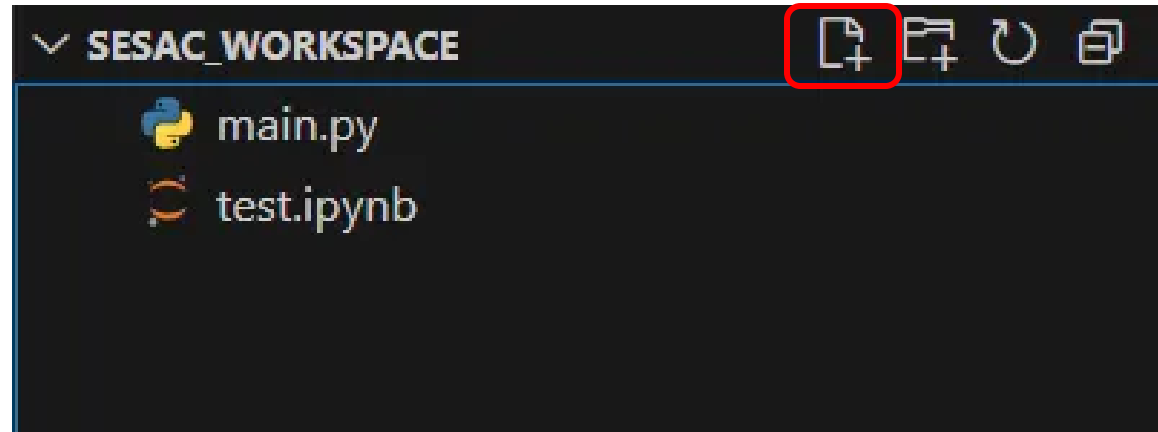


Jupyter Kernel 선택



Kernel 설정 시 지어준 이름 선택

(5) 환경설정 모두 완료 후 테스트 진행



파일 생성하기 (파일이름.확장자 형태)

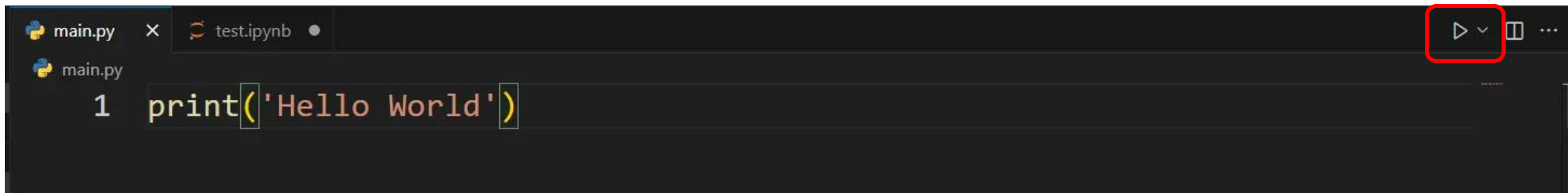
main.py

test.ipynb

(5) 환경설정 모두 완료 후 테스트 진행

main.py

실행 버튼



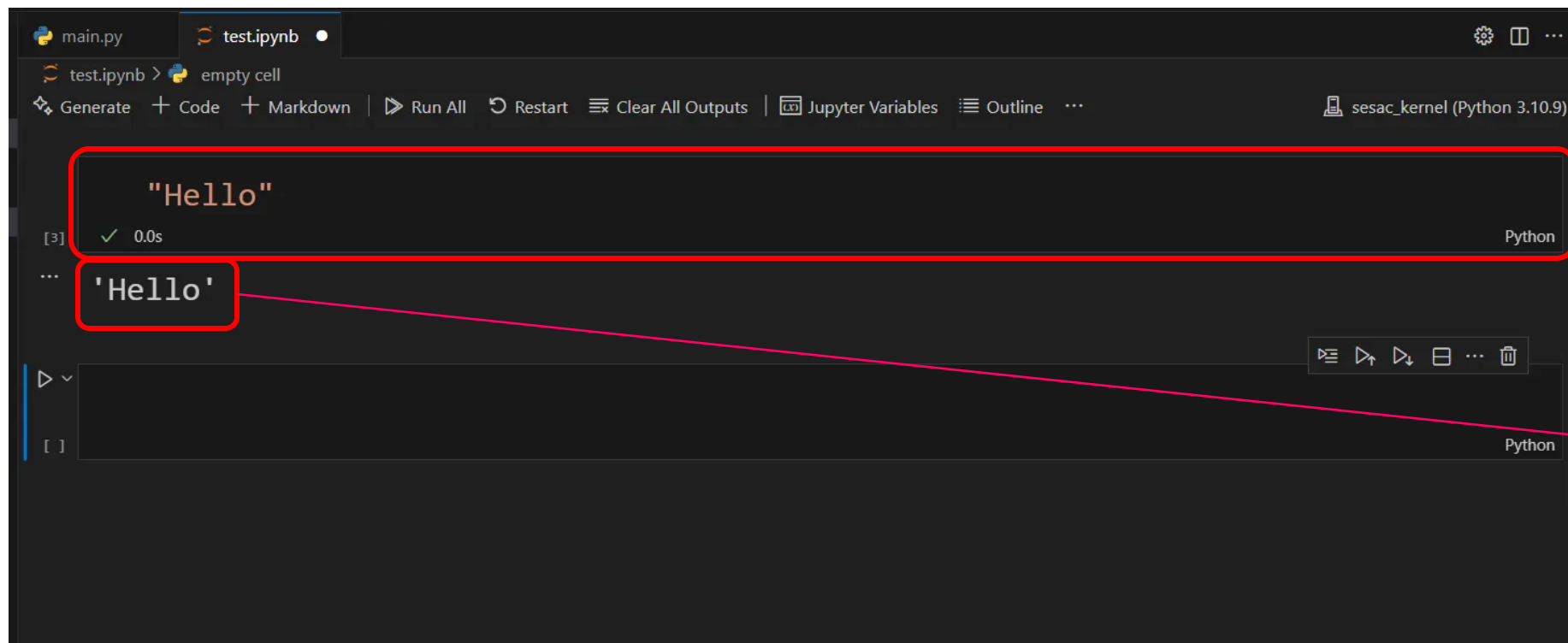
```
main.py x test.ipynb
main.py
1 print('Hello World')
```

```
PS C:\sesac_workspace> & C:/Users/SeoungGun/miniconda3/envs/sesac/python.exe c:/sesac_workspace/main.py
Hello World
```

화면 아래 터미널 창에 결과 출력

(5) 환경설정 모두 완료 후 테스트 진행

test.ipynb



코드 블록

실행 결과

처음에 상단 +Code 버튼 클릭 후

생성된 코드 블록 안에 코드 입력 후
Shift + Enter로 코드 블록 실행

본격적으로 VSCode 환경에서 파이썬을 다뤄봅시다!



기초 자료형과 출력(print)

자료형이란?

데이터 값들의 유형, 종류, 성격을 일컫는다.

구분	종류	예시
숫자 자료형	정수형	35
	실수형	3.141592
	복소수형	3+4j
문자열 자료형	문자열	"Hello World"
논리 자료형	bool	True, False
복합 자료형	List, Tuple, Dictionary, Set	...

To be continued!

숫자(Number) 자료형

숫자로 이루어진 자료형으로, 숫자 간의 연산이 가능하다.

정수형(Integer)

```
15  
-24  
0
```

실수형(floating point)

```
3.141592  
-2.71424  
0.0
```

문자열(String) 자료형

문자나 문자 여러 개가 나열되어 이루어진 자료형

문자열을 표현할 때, 큰 따옴표(" ")나 작은 따옴표(' ') 쌍(pair)로 감싸서 표현한다!

```
"안녕하세요. 이것은 문자열입니다."
```

```
'작은 따옴표로도 동일합니다.'
```

주석(Comment)

프로그램이 실행될 때 무시되는 코드로, 보통 코드에 대한 설명을 작성할 때 주로 사용

1. inline 주석(한 줄 주석)

#을 사용하여 표현

```
# 이 줄은 실제로 실행될 때, 무시가 됩니다!
```

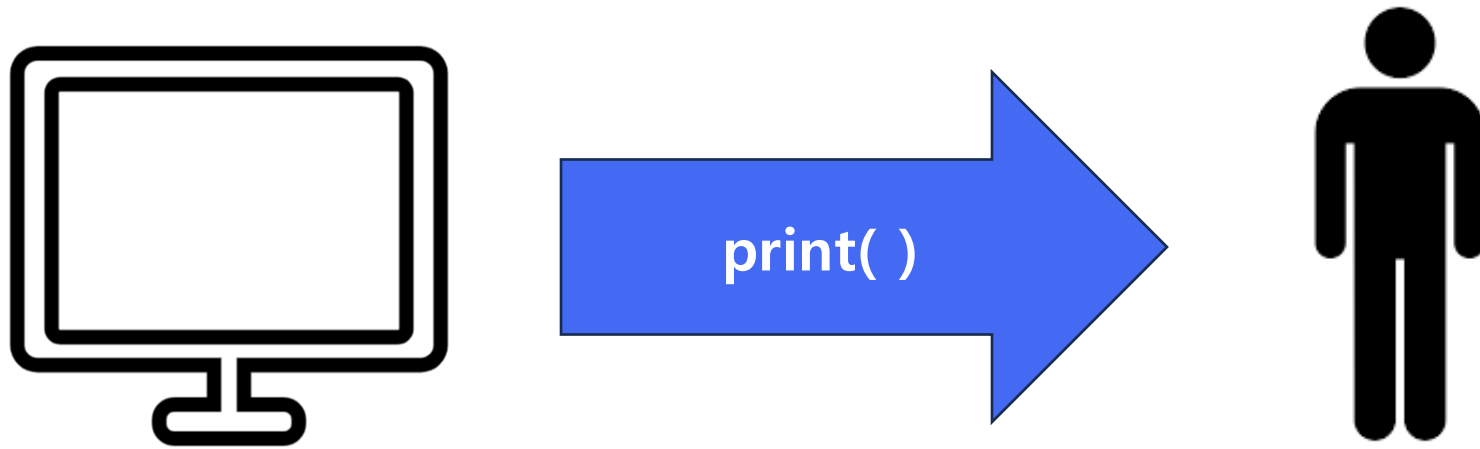
2. multi-line 주석(여러 줄 주석)

작은 따옴표 or 큰 따옴표 3개의 쌍

```
'''  
주석은  
여러 줄에  
걸쳐서도 가능합니다.  
'''
```


출력(print)

컴퓨터로부터 수행된 명령의 결과를 받는 것



원하는 **정보**를 출력 받을 수 있다

출력(print)

출력에 사용되는 print() 함수

```
print(출력할 값)
```

```
print("안녕하세요, 만나서 반갑습니다.")
```

출력
결과

안녕하세요, 만나서 반갑습니다.

여러 개의 값을 출력하려면?

값마다 ,(coma)로 구분하여 출력(값마다 공백 한 칸씩 띄워져서 출력)

```
print("이것은 문자열", -35, 3.14)
```

출력
결과

이것은 문자열 -35 3.14

출력을 여러 줄에 걸쳐서 하려면?

print() 함수를 여러 줄에 걸쳐서 작성! 또는...

```
print("안녕하세요! 삼행시를 지어보려고 해요.")  
print("자유롭게 달리는 두 바퀴 위에서")  
print("전해지는 바람은 시원하고")  
print("거침없이 앞으로 나아간다")
```

출력
결과

```
안녕하세요! 삼행시를 지어보려고 해요.  
자유롭게 달리는 두 바퀴 위에서  
전해지는 바람은 시원하고  
거침없이 앞으로 나아간다
```

이스케이프 시퀀스(Escape Sequence)

문자 그대로 출력하기 어려운 기능을 표현하기 위해 약속된 특수한 문자

₩(백 슬래시) 문자와 함께 사용된다. (특수한 문자이므로 문자열에 속한다)

\와 ₩ 같은 의미

기호	설명
₩n	줄 바꿈(new line) (키보드의 enter 역할)
₩t	수평 탭 (키보드의 tab 역할)
₩b	백 스페이스 (키보드의 backspace 역할)
₩₩	문자 백 슬래시
₩'	문자 작은 따옴표
₩''	문자 큰 따옴표

이스케이프 시퀀스(Escape Sequence)

문자열 안에 **줄바꿈**을 적용해보면?

```
print("자유롭게 달리는 두 바퀴 위에서\n전해지는 바람은 시원하고\n거침없이 앞으로 나아간다")
```

출력
결과

자유롭게 달리는 두 바퀴 위에서
전해지는 바람은 시원하고
거침없이 앞으로 나아간다

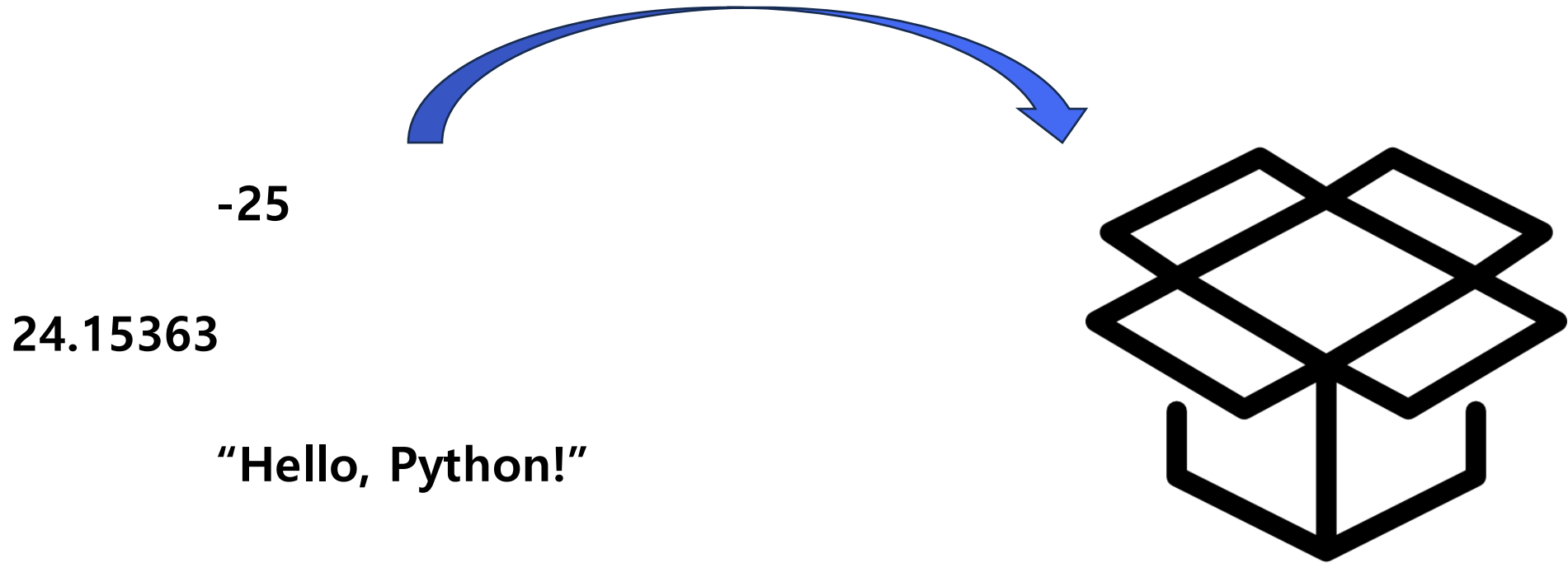


변수(Variable)

변수(Variable)

변수란?

변할 수 있는 값으로, (자료형)값을 **하나**를 보관하는 **이름표**를 가진 보관소.

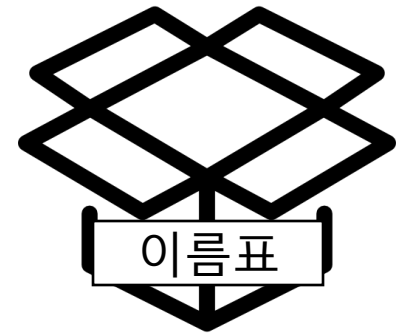


변수(Variable)

변수에 값을 보관하는 방법

변수이름 = (자료형)값

보관소의 이름표 보관할 실제 데이터



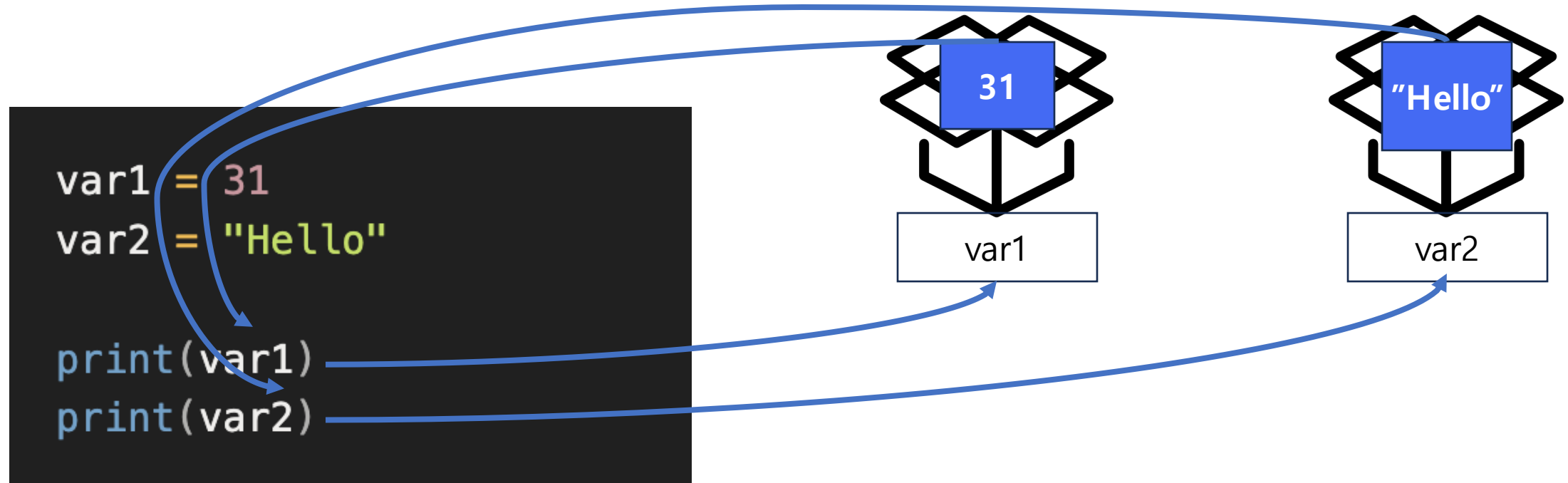
```
num = 15  
name = "Kevin"
```

같다(equal)의 의미가 아닌 할당하다(assign)의 의미

“값을 변수에 할당하다/저장하다/보관하다/대입하다”

변수(Variable)

이름표를 통해 보관된 값을 가져올 수 있다



출력
결과

31
Hello

변수 이름 짓는 규칙 - 반드시 지켜야 하는 규칙(Rule)

```
1name = "Gildong"
```

```
1234 = 5678
```

```
print = 5678  
for = "Hello"
```

```
product name = "Coke"  
product+price = 4000
```

1. 변수 이름이 **숫자**로 시작하면 안됨.

2. **숫자**로만 구성된 변수 이름 금지.

3. 파이썬 문법에서 사용되는 **예약어** 사용 금지

4. 언더스코어(_)을 제외한 **특수문자** 사용 금지

변수 이름 짓는 규칙 - 관례적인 규칙(Naming Convention)

1. 보통 알파벳과 숫자를 조합하여 사용

```
이름변수1 = "Gildong" # 가급적 지양
```

```
name1 = "Gildong" # 알파벳과 숫자의 조합
```

2. 보통 저장되는 값이 의미하는 바를
명사 단어 단위로 표현

```
product_name = "○○"  
phone_number = "01012345678"  
price = 3000  
age = 31
```

3. snake_case 사용

```
pagesize = 15 # 단어끼리 붙이면 가독성 저하  
page_size = 15 # 각 단어끼리 underscore 기호로 구분  
thumbnail_image_url = "https://cdn.example..."
```



자료형의 연산

숫자형(Number) 자료형의 기본 연산자

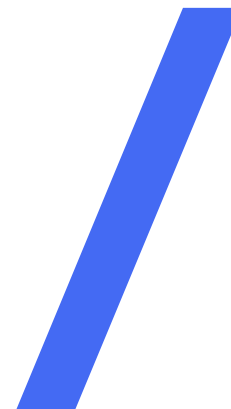
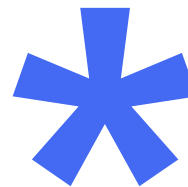
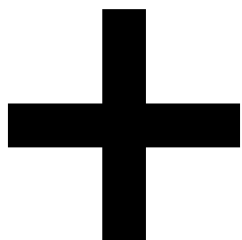
+

-

×

÷

숫자형(Number) 자료형의 기본 연산자



숫자형(Number) 자료형의 기본 연산자

```
print(10 + 10) # 20  
print(10 - 5) # 5  
print(3 * 5) # 15  
print(9 / 4) # 2.25
```

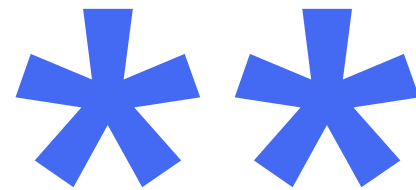

숫자형(Number) 자료형의 특수 연산자



몫 나눗셈 연산자



나머지 연산자



제곱 연산자

숫자형(Number) 자료형의 특수 연산자

```
print(20 // 4) # 5  
print(9 // 4) # 2  
print(17 % 4) # 1  
print(2 ** 4) # 16
```

잠깐! 나머지 연산자(Modular)의 특징?

순환하거나 주기적인 패턴이 띄는 곳에서 주로 활용

ex) 시계 계산, 요일 계산, 리스트 인덱스 순환, 배수

%

3 % 3 -> 0
4 % 3 -> 1
5 % 3 -> 2

6 % 3 -> 0
7 % 3 -> 1
8 % 3 -> 2

9 % 3 -> 0
10 % 3 -> 1
...

```
hour = 23  
after = 5  
new_hour = (hour + after) % 24
```

4

문자열(String) 자료형의 연산자

(1) 문자열 **이어붙이기** 연산자 (Concatenate) **+**

문자열 **+** 문자열

```
print("안녕" + "하세요")  
# 안녕하세요
```

문자열끼리 그대로 이어붙이게 되는 효과

문자열(String) 자료형의 연산자

(2) 문자열 **반복** 연산자 (Repetition) *

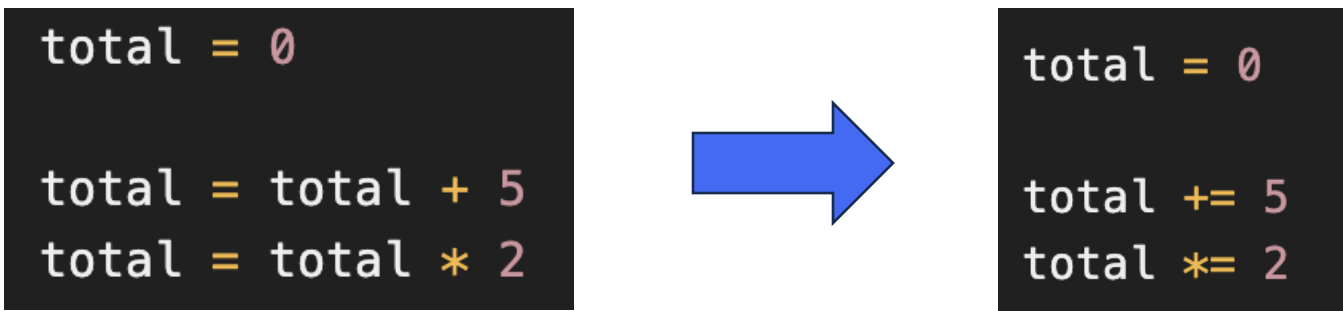
문자열 * 숫자

```
print("안녕" * 3)  
# 안녕안녕안녕
```

숫자의 횟수만큼 반복해서 문자열을 이어붙이게 되는 효과

연산자의 확장

연산 수식 축약 표현: 변수 자기 자신에게 연산을 하는 경우



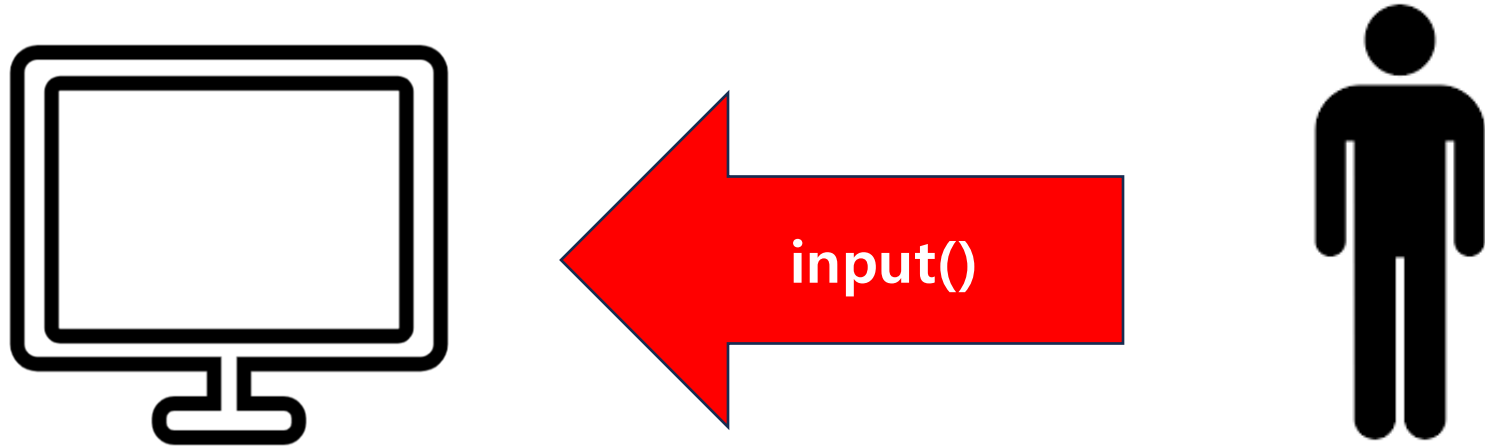
종류	기존 수식	축약 수식
덧셈	$a = a + 1$	$a += 1$
뺄셈	$a = a - 1$	$a -= 1$
곱셈	$a = a * 1$	$a *= 1$
나눗셈	$a = a / 1$	$a /= 1$

다른 연산자($//$, $\%$, $**$ 등등)도 가능하다.



입력과 형 변환(Type Casting)

컴퓨터에게 입력하기

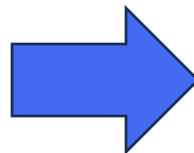


이전에 `print`를 이용하여 컴퓨터로부터 **정보**를 받았다면
정보를 컴퓨터에게 전달할 수 있다

컴퓨터에게 입력하기! input() 함수

사용자에게 **한 줄 단위**로 값을 입력 받음.
입력 받은 값은 **변수**에 보관하여 사용.

```
value = input()
```



```
value = input("값을 입력하세요: ")
```

터미널 창에 커서 부분에 직접 키보드로 입력
(사용자가 입력할 때까지 대기함)



```
Hello World
```

*참고: 함수 인자로 문자열을 넣으면, 해당 문자열이 입력 안내 메시지가 된다.

input() 함수의 특징

무엇을 입력하던 **문자열** 형태로 입력을 받는다.

```
var = input("숫자를 입력하세요:")  
print(var)
```

```
숫자를 입력하세요: 100  
100
```

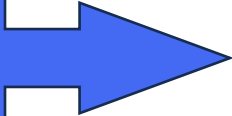
출력되는 값이 숫자일까?

형 변환(Type Casting)

값의 타입을 확인해보자! `type()` 함수

```
int_var = 123  
str_var = "Hello"  
  
print(type(int_var))  
print(type(str_var))
```

출력
결과



```
<class 'int'>  
<class 'str'>
```

형 변환(Type Casting)

형 변환

입력받은 자료형을 해당 자료형답게 사용하기 위해서는 **자료형의 변환**이 필요하다.

```
num1 = input()
num2 = input()

print(num1)
print(num2)
print(num1 + num2)
```

출력
결과

3
8

3
8
38

형 변환(Type Casting)

형 변환 함수

문자열을 정수로, 또는 실수로 바꾸고 싶다면? 자료형을 **형 변환**해줘야함

```
num1 = "1200"  
num2 = "3.14"  
  
num1 = int(num1)  
num2 = float(num2)  
print(num1)  
print(num2)  
  
print(type(num1))  
print(type(num2))
```

출력
결과

```
1200  
3.14  
<class 'int'>  
<class 'float'>
```

형 변환(Type Casting)

형 변환 함수

문자열을 정수로, 또는 실수로 바꾸고 싶다면? 자료형을 **형 변환**해줘야함

형 변환 함수	이름	예시	결과
int()	integer	int("1008")	1008
float()	floating point	float("3.14")	3.14
str()	string	str(2.717)	"2.717"

형 변환(Type Casting)

형 변환 함수

입력 받을 때, 한 번에 형 변환하기

```
int_num = int(input())  
float_num = float(input())  
  
print(int_num)  
print(float_num)  
print(type(int_num))  
print(type(float_num))
```

출력
결과

```
123  
27.45  
  
123  
27.45  
<class 'int'>  
<class 'float'>
```

형 변환(Type Casting)

주의할 점! 변환하고자 하는 값의 타입이 서로 호환이 되어야 한다.

숫자형 문자열 ↔ 숫자형, 숫자형 ↔ 숫자형 등등

```
print(int("I am String!"))
```



```
ValueError: invalid literal for int() with base 10: 'I am String!'
```




논리 자료형과 비교/논리 연산자

논리 자료형

참(True) 또는 거짓(False) 값을 갖는 자료형(bool 타입)

**True는 1, False는 0으로도 간주된다.*

```
print(True)
print(False)
print(type(True))
```

출력
결과

```
True
False
<class 'bool'>
```

논리 자료형

다양한 값들에 대한 **명제** 안에서의 비교를 통해 **논리 자료형의 값**이 결과로 나옴

```
print(3 < 5) # 주어진 명제가 참이면 True  
print(7 == 5) # 주어진 명제가 거짓이면 False  
print(1 >= 5)
```

출력
결과

True
False
False

비교 연산자(Comparison Operators)

명제의 두 값을 비교하여 결과를 **논리 자료형** 형태로 반환

연산자	설명	예시	결과
==	같다	3 == 3	True
!=	같지 않다	3 != 5	True
>	왼쪽이 더 크다	3 > 5	False
<	오른쪽이 더 크다	3 < 5	True
>=	왼쪽이 크거나 같다	3 >= 5	False
<=	오른쪽이 크거나 같다	3 <= 5	False

값 비교를 반드시 숫자형만 가능한 것은 아니다!
문자열, 그리고 그 뿐만 아니라 다른 자료형도 가능하다.

아스키 코드(ASCII Code)

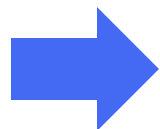
컴퓨터가 문자(알파벳, 숫자, 기호)를 숫자로 약속한 표

문자 **A** = 65

문자 **B** = 66

문자 **a** = 97

문자 **b** = 98



문자 간의 비교는 곧 **아스키 코드**로 비교하는 것

아스키 코드 기반 비교



A는 B보다 작다

a는 b보다 작다

A는 a보다 작다

aaa는 aab보다 작다

aba는 aab보다 작다

*대략 알파벳 간의 **사전순** 비교와 유사함

제어 문자			공백 문자			구두점			숫자			알파벳		
10진	16진	문자	10진	16진	문자	10진	16진	문자	10진	16진	문자	10진	16진	문자
0	0x00	NUL	32	0x20	SP	64	0x40	A	96	0x60				
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a			
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b			
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c			
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d			
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e			
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f			
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g			
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h			
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i			
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j			
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k			
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l			
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m			
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n			
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o			
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p			
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q			
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r			
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s			
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t			
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u			
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v			
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w			
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x			
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y			
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z			
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{			
28	0x1C	FS	60	0x3C	<	92	0x5C	\	124	0x7C				
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}			
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~			
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL			

논리 연산자(Logical Operators)

여러 조건을 동시에 확인해야할 때, 복잡한 조건을 표현할 때 사용되는 연산자

연산자	설명	예시	결과
and	두 명제가 모두 참이어야 참	True and True	True
or	두 명제 중 하나라도 참이면 참	True or False	True
not	명제를 뒤집음	not True	False

논리 연산자(Logical Operators) - **and** 연산자

두(또는 그 이상) 명제가 **모두 참**이어야 참이 되는 연산자

경우	결과
True and True	True
True and False	False
False and True	False
False and False	False

```
print(3==3 and 4<=5 and 6>2)
```

- **3==3**: 3은 3과 같은가요? **True**
- **4<=5**: 4는 5보다 작거나 같은가요? **True**
- **6>2**: 6은 2보다 큰가요? **True**

모든 명제가 참이므로 결과는 True!

and 연산자는 ~이면서 / ~그리고 / ~동시에 라고 표현할 수 있다.

논리 연산자(Logical Operators) - or 연산자

두(또는 그 이상) 명제 중 **하나라도 참**이면 참이 되는 연산자

경우	결과
True or True	True
True or False	True
False or True	True
False or False	False

```
print(3==4 or 4<=5 or 6<2)
```

- 3==4: 3은 4와 같은가요? False
- 4<=5: 4는 5보다 작거나 같은가요? True
- 6<2: 6은 2보다 작은가요? False

하나의 명제(두번째)가 참이므로 결과는 True!

or 연산자는 ~또는 / ~이거나 라고 표현할 수 있다.

논리 연산자(Logical Operators) - not 연산자

명제의 결과를 뒤집음

```
print(not 3==4)
```

3은 4와 같은 것이 아니다 → True

False

```
print(not 10!=2)
```

10과 2는 같지 않은 것이 아니다 → False

True

연산의 우선순위

우선순위	연산자
1	()
2	**
3	*, /, %, //
4	+, -
5	>, >=, <, <=
6	==, !=
7	and, or, not

연산의 우선순위 (복합 연산)

```
result = (3 + 4) * 2 > 10
```

1. 괄호에 의해 $3 + 4$
2. 곱셈에 의해 $7 * 2$
3. $>$ 에 의해 $14 > 10$

True

```
a = 5  
b = 2  
c = 10  
result = (a * b > 8) and (c // b == 5)
```

1. 첫번째 괄호의 $a * b$
2. 첫번째 괄호의 $10 > 8$
3. 두번째 괄호의 $c // b$
4. 두번째 괄호의 $5 == 5$
5. and 연산의 True and True

True

감사합니다!